

基于水热耦合模型的药材烘干过程模拟与时间预测

摘要

干燥是决定中药材成品品质的关键工序之一。针对药材烘干过程中温湿场演化及失水收缩问题, 本文将建立由常热物性传递、非线性水热物性耦合到收缩移动边界逐步扩展的模型体系。实现烘干过程模拟与时间预测, 并分析端面传递及蒸发潜热的影响。

针对问题一: 首先采用分段线性插值重构逐秒烘房环境边界, 基于 **Fourier 导热定律**与 **Fick 扩散定律**, 建立固定半径圆柱的一维径向常热物性热湿同步传递模型, 数值求解上采用有限体积法、隐式时间格式与 **Picard 迭代**。结果表明, 30 min 时中心与表面温度分别为 **33.5756 °C** 和 **36.7857 °C**, 水分浓度分别为 **2.5500 kg/kg** 和 **1.5102 kg/kg**; 距表面 0.5 cm 处的水分降幅仅为表面的 16.8%, 说明热量已传入中心, 而明显失水主要集中于表层, **内部水分迁移显著滞后于热量传播**。

针对问题二: 我们把第一问的热湿同步计算扩展为**温度、水分通过物性相互影响的非线性耦合模型**, 在每个时间步内依次更新热物性、温度场、扩散系数和水分场。结果表明, 3 h 时中心与表面温度温差仅为 0.1169 °C; 同期水分浓度差值仍达 0.7581 kg/kg, 体积平均水分浓度下降约 45.78%。这表明药材虽已基本实现温度均匀, 内部仍保留明显水分梯度, 升温完成并不意味着干燥完成, 后续干燥主要受内部水分扩散限制。

针对问题三: 沿用问题二的水热耦合模型, 以第 3 ~ 4 h 环境数据的时间**加权均值**延拓后续边界, 建立全过程干燥时间预测模型。以全域最大水分浓度首次严格低于 0.15 kg/kg 为终止条件, 采用自适应分段时间步推进平衡精度与计算量。计算得到烘干时间为 **57.5047 h**, 药材中心最终控制整体干燥终点; 表面比中心提前约 44.94 h 达标, 表明表层干燥后内部仍需持续排湿, 依据表面状态判断结束将显著低估烘干时长。而且干燥过程还呈现前期较快、后期拖尾的特征。

针对问题四: 针对问题四, 我们在问题三的模型基础上, 引入材料坐标 $\xi = r/R(t)$, 将移动区域映射至固定计算区间, 结合附录 4 变物性关系建立**水热耦合收缩移动边界模型**, 在每个时间步更新控制体几何和物性参数, 并沿用第三问的终点判断依据。计算得到烘干时间为 **51.1172 h**, 较恒定半径 129.8869 h 缩短 60.64 %, 反映收缩几何与物性变化的综合影响。药材半径在 35 h 稳定至 1.2 cm 时, 中心水分仍为 0.1975 kg/kg 并不达标, 说明尺寸稳定不能替代干燥终点判据。

最后, 我们还针对问题三和问题四进行了模型拓展, 为验证假设合理性, 我们重新考虑两端面水热交换对结果的影响情况。结果表明二维结果分别比一维主结果分别增加 119 s 和 30 s, 相对差异均小于 0.06%。说明基本假设合理。同时引入蒸发潜热的影响, 蒸发导致两问分别延长 3.0458 h、5.30%, 5.3036 h、10.38%。说明蒸发耗热可能通过降低温度、减弱水分扩散而推迟达标, 其定量影响仍需实验校准。

关键字: 药材烘干; 水热耦合; 有限体积法; 移动边界; 干燥时间预测

一、问题重述

1.1 问题背景

中药材采收后通常含有较多水分，若不能及时脱水，储运过程中容易发生霉变和品质劣化；但干燥强度过大或时间过长，又可能造成有效成分损失、外观收缩及额外能耗。与易受天气制约的自然晾晒相比，热风烘干便于调节温度和湿度，更适合规模化生产，但仍存在内部干燥不均、能耗偏高和终点难以准确判定等实际问题，现有热风装备仍有进一步完善机理模型和控制方法的需求 [1]。学者在研究中发现，药材在干燥过程中，水分由内部向外部迁移 [2]，这说明仅凭整体平均含水率难以反映药材的真实干燥程度，需要同时刻画其内部温度场、水分场及几何尺度的动态变化，并据此确定可靠的干燥终点。

1.2 问题提出

研究对象为一根长度为 25 cm、初始半径为 2 cm 的近圆柱形药材，其初始温度和干基含水率分别为 28 °C 和 2.55 kg/kg。已知烘干前期烘炉内环境的温湿变化与干燥过程中药材的半径记录，以及不同阶段下热物性和水分扩散系数的经验关系。需要在给定数据时段、输出精度和采样间隔的约束下，依次完成以下任务：

问题一：针对开始烘干后的 1800 s，依据随时间变化的外部环境和给定常物性参数，描述热量与水分沿药材半径方向的传递过程，求得各时刻、各径向位置的温度和水分浓度，并按指定节点列示结果，同时生成时间间隔 1 s、空间间隔 0.1 cm 的完整数据。

问题二将研究范围由短时预热扩展至预热与稳定干燥组成的全过程。考虑密度、比热容、导热系数和有效扩散系数随温度及含水状态变化所形成的非线性联系，确定前 3 h 内温度场和水分场的演化，并按规定时间与距离要求输出完整结果。

问题三：在半径暂视为不变的条件下，把水分迁移模型延伸至数日尺度。以药材任意位置的水分浓度均低于 0.15 kg/kg 为统一达标准则，寻找首次满足该准则的时刻，作为固定尺寸下的烘干时长，并给出每隔 60 s 的完整径向水分分布。由于环境观测只覆盖烘干初期，长期边界需在现有数据基础上作合理延拓。

问题四：引入失水导致的半径变化，将固定区域上的传递问题改写为随时间收缩的空间区域，并采用相应经验参数重新计算水分分布与达标时刻，进而分析几何收缩对烘干时长的修正作用；最终导出 60 s 时间间隔和 0.1 cm 径向间隔的水分分布结果。

二、问题分析

药材的两个端面相较于主体部分较小，为突出主要传递方向，我们忽略端面与环境之间的热量和水分交换，并在问题一至三中将半径视为常量，只考察由圆柱表面指向中心的径向温度与水分变化。附件给出的环境数据是离散记录，计算前需通过插值得到逐秒数据；随后分别描述径向热传导和水分扩散过程，并以表面对流边界反映药材与烘房环境之间的有限速率交换。

四个问题沿同一烘干过程逐层深入。问题一研究固定物性条件下的短时预热；问题二引入随温度和含水率变化的材料物性；问题三把计算延伸至完整烘干周期；问题四再考虑失水收缩造成的区域变化。由此，我们采用统一的传热传质框架，并根据各问特点依次加入物性变化、长期环境边界和尺寸收缩等因素。

2.1 问题一的分析

问题一关注烘干初期预热平衡阶段药材内部温度与水分的时空变化。药材内部同时发生热传导和水分扩散，温度场与水分场应分别描述。附录 2 所给物性关系中不存在温度与水分之间的交叉影响，故两种传递过程虽共用几何区域和环境条件，却不会相互反馈，可以独立求解后统一输出。

2.2 问题二的分析

问题二把研究时段由短时预热延伸至预热平衡和恒温干燥阶段。此时，材料的物理性质会随含水率改变，水分扩散系数又同时取决于温度和含水率。温度场的变化会影响水分迁移，含水率的下降也会反过来改变传热参数，两者由此形成非线性水热耦合。建模时还要注意核对温度单位和各参数的适用范围，避免因单位混用造成计算偏差。

2.3 问题三的分析

问题三要确定药材达到烘干标准的时刻，其实质是求解带有全域含水率约束的终止时间。由于实际烘干过程可能超过附件 1 的实测时段，附件数据结束后的烘房环境必须作出合理描述。达标条件要求药材各处含水率均低于 0.15 kg/kg ，判断时要检查整个区域，并重点关注药材中心，即水分最难散失的位置。端面暴露与蒸发吸热也可能改变最终烘干时间，需对其影响作补充讨论。

2.4 问题四的分析

问题四把药材失水引起的体积收缩纳入计算。半径随含水率减小后，内部传热传质距离随之缩短，药材表面积和内部物性也会发生变化，原先定义在固定区域上的传递问

题因而转化为移动边界问题。我们需要建立药材尺寸与含水率之间的关系，并在不断变化的区域内追踪温度和水分布。对于题目指定的物理位置，还应先判断其在收缩过程中是否仍处于药材内部，再依据全域含水率标准确定最终烘干时间。

三、模型假设

- 1. 药材近似为轴对称圆柱。烘干开始时，药材内部温度和水分浓度均匀分布。
- 2. 忽略端面附近的局部效应，认为药材热量和水分只沿径向迁移并通过侧面交换。
- 3. 将药材假设为连续、均匀且各向同性的多孔介质，烘干过程中不发生开裂、局部塌陷或组织突变。
- 4. 核心模型下各问题忽略蒸发潜热对药材的影响。
- 5. 药材表面仅通过对流形式与烘房交换热量和水分。
- 6. 干燥过程中忽略干物质的挥发、分解和流失，干物质总质量保持不变。
- 7. 问题一至问题三中药材尺寸保持不变，问题四假设药材收缩沿径向均匀发生。

四、符号说明

表 1 模型中主要符号说明

符号	含义	单位
L	药材的轴向长度	m
t	烘干时间	s
r	药材内部某点到圆柱中心轴线的当前距离	m
$R_0, R(t)$	药材的初始半径和时变半径	m
$\xi = r/R(t)$	均匀收缩模型中的归一化坐标	-
$T(r, t)$	时刻 t 、径向位置 r 处的药材温度	°C
$T_K(r, t)$	药材的绝对温度, $T_K = T + 273.15$	K
$T_0(t), T_s(t)$	烘房空气温度和药材真实表面温度	°C
$C(r, t)$	时刻 t 、径向位置 r 处的药材干基水分浓度	kg/kg
$C_0(t), C_s(t)$	烘房等效水分浓度和药材真实表面水分浓度	kg/kg
C_d	干燥判定阈值, 恒为 0.15	kg/kg
$\rho(C)$	药材密度	kg/m ³
$c_p(C)$	药材比热容	J/(kg · K)
$k(C)$	热传导系数	W/(m · K)
$D(C, T_K)$	水分浓度扩散系数	m ² /s
h_T	对流换热系数	W/(m ² · K)
h_m	对流传质系数	m/s
t_d	药材达到干燥要求的最早时刻	s

五、模型的建立与求解

5.1 数据预处理

5.1.1 数据检验

对附件 1 和附件 2 进行缺失值、重复记录、异常值及单调性检验。结果表明，两组数据均无明显缺失或重复等异常现象，附件 1 中的温度和水分浓度均呈先升高后趋稳的变化特征，附件 2 中的药材半径随时间单调不增。因此，保留原始数据，不进行删除、补缺或平滑处理。

5.1.2 单位统一

由于水分浓度扩散系数的计算采用热力学温度，将已有附件中的摄氏温度数据转换为开尔文温度数据：

$$T_K = T_C + 273.15K \quad (1)$$

同时，将药材半径由厘米换算为米对齐附录经验公式要求，并在计算将温度重新转为摄氏度温度做直观可视化图表。

5.1.3 数据可视化

分别绘制烘房温度、水分浓度及药材半径的时间序列图。对检验后的附件 1 数据进行时序观察可知，烘房温度和水分浓度在前期持续上升，随后逐渐进入相对稳定状态，整体变化连续，未出现明显的孤立突变。

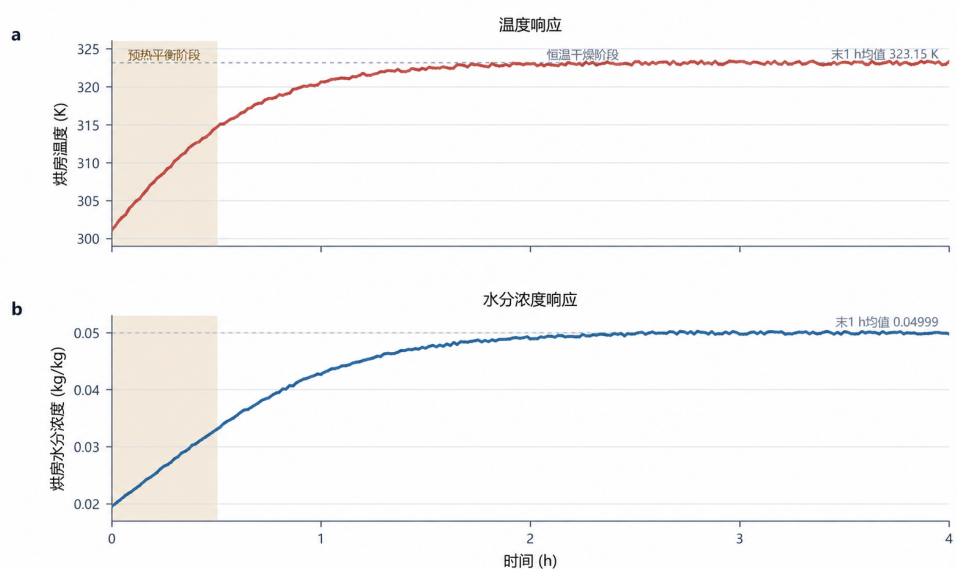


图 1 烘房温度与水分浓度随时间的变化

进一步考察附件 2 中的半径数据可以发现，药材半径在烘干初期下降较快，随后收缩速率逐渐减小并趋于稳定，说明尺寸变化具有明显的阶段性，因此在问题四中的半径收缩情境下要考虑网格尺寸的修正。

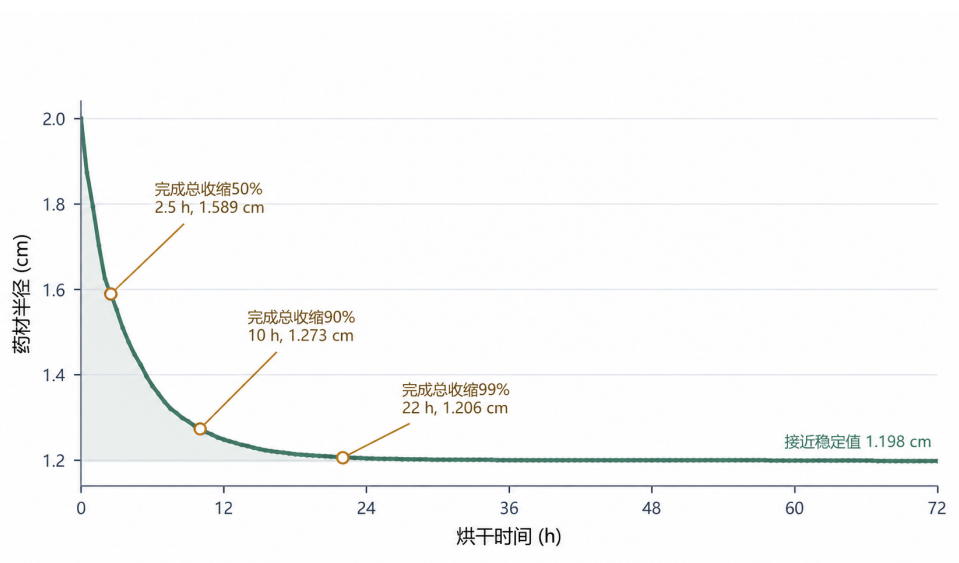


图 2 药材半径随烘干时间的变化

5.2 问题一模型的建立与求解

5.2.1 建模思路与边界数据处理

药材长 $L = 25\text{ cm}$ ，半径 $R = 2\text{ cm}$ ，其侧表面积与两端面积之比为

$$\frac{A_{\text{side}}}{A_{\text{end}}} = \frac{2\pi RL}{2\pi R^2} = \frac{L}{R} = 12.5.$$

由此忽略端部效应，将药材视为均匀、各向同性的长圆柱，只保留径向温湿变化，考察热量与水分沿径向的迁移。问题一不计水分相变潜热，且扩散系数 D 仅随水分浓度 C 改变，因此温度场和水分场之间不存在状态量反馈，可分别求解。

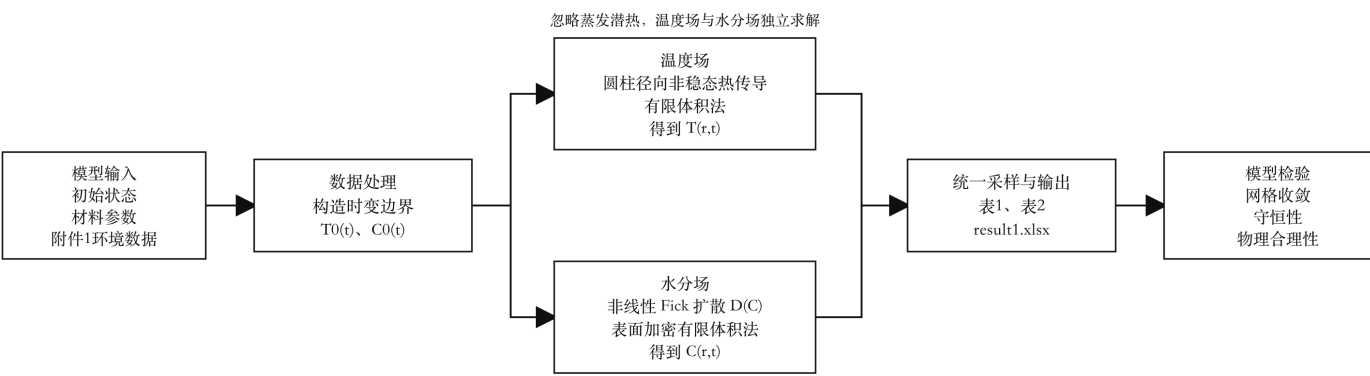


图 3 问题一求解流程

附件 1 以 60 s 为间隔记录烘房温度和水分浓度，而控制方程需要逐时间步更新环境边界。记 $y \in \{T_0, C_0\}$ ，对于相邻观测点 (t_j, y_j) 和 (t_{j+1}, y_{j+1}) ，在 $t_j \leq t \leq t_{j+1}$ 内采用分段线性插值：

$$y(t) = (1 - \theta)y_j + \theta y_{j+1}, \quad \theta = \frac{t - t_j}{t_{j+1} - t_j}. \quad (2)$$

根据附件一上式有 $t_{j+1} - t_j = 60 \text{ s}$ 。且 $T_0(t)$ 和 $C_0(t)$ 是烘房环境量，并非药材表面状态；药材表面温度和水分浓度需要通过边界条件计算确定。

5.2.2 温度场与水分场模型

(1) 温度场控制方程 取半径为 r 、厚度为 dr 的环形微元。问题一只考虑药材的显热积累，径向热流服从傅里叶导热定律。对该微元列写能量守恒，得到

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(kr \frac{\partial T}{\partial r} \right), \quad 0 < r < R. \quad (3)$$

药材初始条件下各处水分浓度与温度均匀，圆柱轴线上不考虑导热与对流，侧面与炉内空气发生对流换热。相应的初始条件、中心对称条件和表面对流条件为

$$\begin{cases} T(r, 0) = 28^\circ\text{C} \\ \frac{\partial T}{\partial r} \Big|_{r=0} = 0 \\ k \frac{\partial T}{\partial r} \Big|_{r=R} = h_T [T_0(t) - T(R, t)] \end{cases} \quad (4)$$

其中药材密度 $\rho = 820 \text{ kg/m}^3$ ，药材比热容 $c_p = 2600 \text{ J/(kg} \cdot \text{K)}$ ，热传导系数 $k = 0.36 \text{ W/(m} \cdot \text{K)}$ ，对流换热系数 $h_T = 25 \text{ W/(m}^2 \cdot \text{K)}$ ，最大半径 $R = 0.02 \text{ m}$ 。方程右端只含内部导热项。由于本问忽略水分蒸发潜热，水分迁移项不计入能量平衡等式，温度场方程可单独求解。

(2) 水分场控制方程 对同一环形微元列写水分守恒方程式，并用 *Fick* 定律描述药材内部的径向扩散，可得

$$\begin{cases} \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C) \frac{\partial C}{\partial r} \right], 0 < r < R \\ D(C) = 7 \times 10^{-9} \exp \left(-\frac{0.89}{C} \right) \end{cases} \quad (5)$$

对于药材表面的质对流方程，根据题目初始状态下水分浓度在药材各处均匀，轴线处满足对称条件，同时药材表面的水分扩散通量与空气侧的对流传质通量相等，得到水

分浓度的表面方程

$$\begin{cases} C(r, 0) = 2.55 \text{ kg/kg} \\ \frac{\partial C}{\partial r} \big|_{r=0} = 0 \\ -D(C) \frac{\partial C}{\partial r} \big|_{r=R} = h_m [C(R, t) - C_0(t)] \end{cases} \quad (6)$$

其中 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。边界式左端表示药材由内向外的水分扩散通量，右端表示表面与环境之间的传质通量。方程式中 $D(C)$ 中不受温度变量影响，因此本问实际上可以并列处理同一环境记录下温度和水分浓度的瞬态传递问题，不存在水热耦合关系。对于含有 $D(C)$ 的非线性扩散方程，需要在接下来的有限体积离散模型下迭代求解。

5.2.3 有限体积离散与数值求解

Vu 和 Tsotsas 的研究表明，有限体积法能够在离散层面保持多孔介质干燥过程的质量与能量守恒 [3]。因此，本文沿径向划分环形控制体。按单位轴长计，第 i 个控制体的体积和界面面积为

$$V_i = \pi \left(r_{i+\frac{1}{2}}^2 - r_{i-\frac{1}{2}}^2 \right), A_{i\pm\frac{1}{2}} = 2\pi r_{i\pm\frac{1}{2}} \quad (7)$$

令 ψ 表示温度 T 或水分浓度 C ，相应地取 $(S, \Gamma) = (\rho c_p, k)$ 或 $(1, D)$ 。采用后向 Euler 格式后，第 i 个控制体满足

$$S_i V_i \frac{\psi_i^{n+1} - \psi_i^n}{\Delta t} = G_{i+\frac{1}{2}} (\psi_{i+1}^{n+1} - \psi_i^{n+1}) - G_{i-\frac{1}{2}} (\psi_i^{n+1} - \psi_{i-1}^{n+1}), \quad (8)$$

其中界面系数 G 按相邻半控制体的传递阻力串联计算。中心界面面积为零，故对称边界自然满足；外表面则将内部半控制体阻力与空气侧对流阻力串联，以保证界面通量连续。离散后得到三对角线性方程组，并采用 Thomas 追赶法求解。

温度场采用均匀网格 $\Delta r_T = 0.025 \text{ cm}$ 和时间步 $\Delta t_T = 0.25 \text{ s}$ 。考虑水分梯度集中于表层，水分场采用映射

$$r_{j+\frac{1}{2}} = R \left[1 - \left(1 - \frac{j}{N_C} \right)^2 \right] \quad (9)$$

进行网格划分。水分方程中的 $D(C)$ 通过 Picard 迭代处理：每轮以当前浓度更新 D ，重新组装方程并用 Thomas 法求解，直至 $\|C^{n+1,(m+1)} - C^{n+1,(m)}\|_\infty < 10^{-10}$ 。

有限体积未知量位于控制体中心。为得到题目要求的真实表面浓度，根据表面扩散通量与空气侧对流传质通量连续，重构

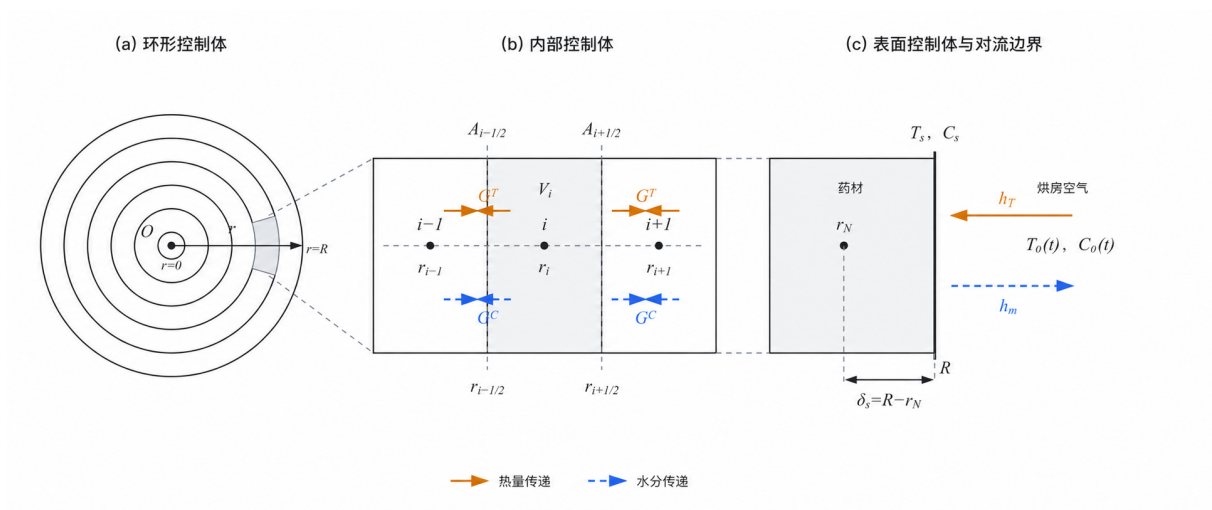


图4 圆柱径向有限网格及表面对流边界示意图

$$C_s = \frac{\frac{D_N}{R - r_N} C_N + h_m C_0}{\frac{D_N}{R - r_N} + h_m} \quad (10)$$

最终每 1 s 保存一次结果，并插值得到径向间隔为 0.1 cm 的输出，问题一每秒计算结果详见”result1.xlsx”。

5.2.4 求解结果与分析

问题一要求的各时刻和径向位置温度如表2所示。

表2 30 min 内药材的温度（单位：℃）

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0001	28.0004	28.0041	28.0328	28.1801
300	28.0410	28.0637	28.1516	28.3683	28.8490
600	28.4538	28.5364	28.8043	29.3162	30.1654
900	29.3248	29.4587	29.8758	30.6164	31.7306
1200	30.5432	30.7102	31.2227	32.1129	33.4278
1500	31.9961	32.1871	32.7664	33.7465	35.1204
1800	33.5756	33.7723	34.3644	35.3624	36.7857

到 1800 s 时,炉温升至 41.5130℃,药材表面和中心温度分别为 36.7857℃ 与 33.5756℃,相差 3.2101℃。各时刻的温度均由中心向表面递增,外表面响应较快,中心升温仍有明显滞后。30 min 内药材尚未达到内外等温状态。

相同时刻的水分浓度见表3。

表 3 30 min 内药材的水分浓度 (单位: kg/kg)

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5352	1.8769
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4645	1.6585
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3754	1.5102

水分变化比温度变化更集中于表层。至 1800 s，表面浓度降至 1.5102 kg/kg，中心仍保持初始值 2.55 kg/kg，截面平均浓度为 2.2936 kg/kg。这说明药材虽已开始失水，30 min 预热却不足以使水分扰动深入中心区域。

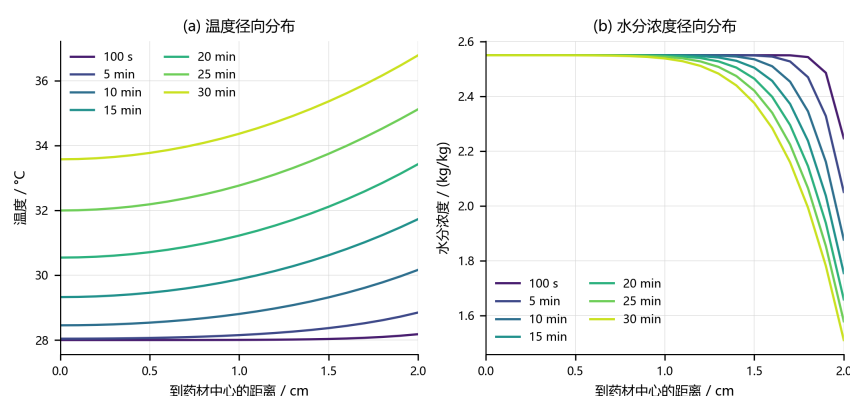


图 5 不同时刻药材温度与水分浓度的径向分布

图5将不同时刻的径向剖面置于同一坐标系内。温度曲线随时间整体上移，梯度逐渐向中心传递；水分曲线的主要变化则长期停留在外缘，内部大部分区域仍接近初始浓度。两组剖面反映出不同的传递时间尺度：本阶段热量向内传播较快，水分迁移仍受内部扩散限制。

为比较预热阶段热量与水分的径向传递过程，我们将计算得到的温度场和水分场绘制成时空云图，如图6所示。图中横轴为时间，纵轴为到药材中心的距离，颜色分别表示局部温度与水分浓度。

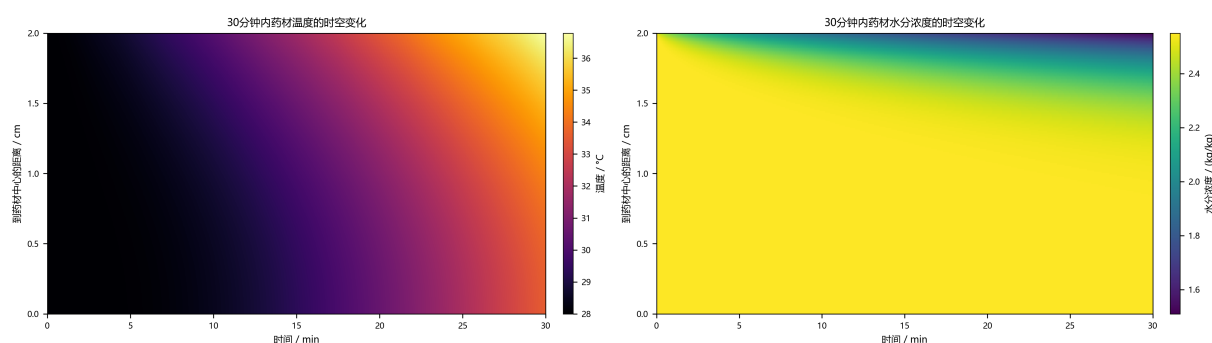


图 6 30 min 内药材温度与水分浓度的时空分布 (左: 温度场; 右: 水分场)

图6左图中，药材温度随加热时间持续升高，靠近外表面的区域响应较快，温度变化随后逐渐向中心传递。右图呈现出不同的变化特征：水分浓度的下降主要集中在外表面附近，内部大部分区域在 30 min 内仍接近初始状态。两幅云图的对比说明，热量已在整个径向区域内传播，而水分迁移仍受到内部扩散的限制。

5.2.5 模型检验

(1) 插值方法敏感性分析 为考察边界数据处理方法对计算结果的影响，我们分别采用分段线性插值、分段三次 Hermite 插值 (PCHIP) 和端点约束拉伸指数模型构造 0 ~ 1800 s 内的连续边界函数，并对三种方法的拟合精度、留一预测误差及曲线形状进行比较。结果如图7。

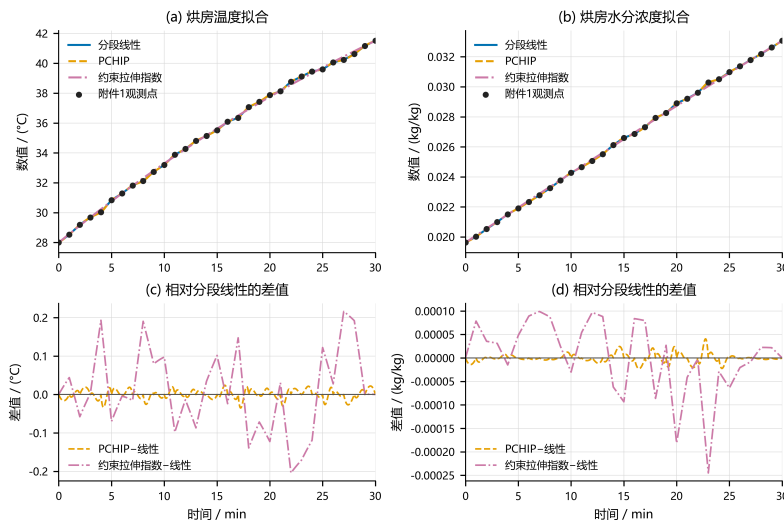


图 7 三种边界构造方法敏感性分析

三种方法所得曲线高度一致，温度和水分浓度的最大方法间差异分别仅占观测变化幅度的 1.74% 和 1.85%。考虑到分段线性插值严格通过全部观测点、无过冲且计算简单，本文采用其构造逐秒边界条件。

(2) 参数变化灵敏度分析 本文以参数变化正负 20% 时温升或失水量的最大相对变化作为灵敏度指标。计算结果表明，参数增加 0.2 倍情况下密度和比热容的影响均为 19.97%，对流传质系数为 14.75%，对流换热系数为 10.59%，导热系数为 10.00%，扩散前因子为 7.21%，扩散指数系数为 3.00%。因此，参数影响程度满足 $\rho c_p > h_m > h > k > D_0 > \beta$ 。

其中，密度和比热容通过乘积 ρc_p 共同决定药材的体积热容，二者不属于相互独立的敏感因素。温度模型主要受体积热容控制，水分模型主要受对流传质系数控制，而扩散指数系数在考察范围内影响较小，灵敏度分析结果详见支撑材料“A-problem1-sensitivity”。

5.3 问题二模型的建立与求解

问题二采用附录 3 中的经验公式。水分变化会改变药材的密度、比热容和导热系数，温度和水分又共同影响有效扩散系数，温度场与水分场方程因此存在耦合关系。我们以附件 1 的烘房温湿数据作为时变边界，建立固定半径圆柱的一维径向热湿耦合模型，求解前 3 h 内药材的温度场和水分场。

5.3.1 热—湿耦合模型的建立

本问计算范围为 $0 \sim 3\text{ h}$ ，其间半径保持不变；药材内部没有宏观对流和体积热源，热量按傅里叶定律传递，水分按有效扩散规律迁移。表面分别满足对流换热和对流传质条件，取 $h_T = 25\text{ W}/(\text{m}^2 \cdot \text{K})$, $h_m = 8.0 \times 10^{-7}\text{ m/s}$ 。

在半径 r 处取厚度为 $\text{d}r$ 、轴向长度为 1 的圆环微元，其体积为 $\text{d}V = 2\pi r \text{d}r$ 。热通量和水分通量分别为

$$q_r = -k(C) \frac{\partial T}{\partial r}, \quad J_r = -D(C, T) \frac{\partial C}{\partial r}.$$

对微元列能量守恒，并代入热通量后，得到温度控制方程

$$\rho(C)c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[rk(C) \frac{\partial T}{\partial r} \right] \quad (11)$$

同理，由圆环微元内水分的积累量等于径向净流入量，可得

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[rD(C, T) \frac{\partial C}{\partial r} \right] \quad (12)$$

式中 $k(C)$ 与 $D(C, T)$ 均随局部状态变化，因而须保留在空间微分算子内部。附录 3 给出的物性关系整理为表。

表 4 药材参数计算公式

物性参数	计算公式
密度	$\rho(C) = 650 + 128C$
比热容	$c_p(C) = 1450 + \frac{2736C}{C+1}$
导热系数	$k(C) = 0.21 + \frac{0.38C}{C+1}$
有效扩散系数	$D(C, T_K) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T_K}\right)$

其中 $T_K = T + 273.15\text{ K}$ 。水分场通过 $\rho(C)$ 、 $c_p(C)$ 和 $k(C)$ 影响温度场；求得的新温度又与当前水分共同决定 $D(T, C)$ 。干燥开始时，药材内部温度和水分均匀： $T(r, 0) = 28^\circ\text{C}$, $C(r, 0) = 2.55\text{ kg/kg}$ 。圆柱中心满足对称条件，同时在 $r = R$ 处，内部传递通量与

表面对流通量连续，有以下热质耦合方程组：

$$\begin{cases} \frac{\partial T}{\partial r}\big|_{r=0} = 0, & \frac{\partial C}{\partial r}\big|_{r=0} = 0. \\ k_s \frac{\partial T}{\partial r}\big|_{r=R} = h_T [T_a(t) - T_s(t)] \\ -D_s \frac{\partial C}{\partial r}\big|_{r=R} = h_m [C_s(t) - C_a(t)]. \end{cases} \quad (13)$$

这两个边界均为第三类 Robin 边界，考虑到热质对流情况，要用上述对流方程组求解每 1s 药材表面温度，本文沿用问题一的分段线性插值拟合方法，第二问插值终点为 3 h，始终位于附件一数据的覆盖范围内，不涉及接下来的数据预测。

5.3.2 有限体积离散与耦合求解

第二问沿用问题一的环形有限体积框架。温度场和水分场均采用向表面加密的单元中心网格，详见公式 (9)。

同问题一类似，圆心和表面输出分别根据对称性与通量连续关系重构，内部界面按相邻半控制体的传递阻力串联，外表面按内部半单元阻力与对流阻力串联，然后进行迭代。

与问题一相比，本问引入了药材的参数随状态 T 和 C 变化。每个时间步以 T^n 和 C^n 为初值进行分块 Picard 迭代：第 m 轮先根据 $C^{(m)}$ 更新 ρ 、 c_p 和 k ，求得 $T^{(m+1)}$ ；再由 $C^{(m)}$ 和新温度 $T^{(m+1)}$ 更新 D ，求得 $C^{(m+1)}$ 。当

$$\left\| T^{(m+1)} - T^{(m)} \right\|_{\infty} < 10^{-8} \text{ }^{\circ}\text{C}, \quad \left\| C^{(m+1)} - C^{(m)} \right\|_{\infty} < 10^{-10} \text{ kg/kg}$$

同时成立时接受当前时间步。正式计算取 $N = 160$ 、 $\Delta t = 0.5 \text{ s}$ ，每 1 s 保存一次结果，问题二详细结果见“result2.xlsx”。

5.3.3 计算结果与分析

前 3 h 内各指定位置的温度计算结果如下。

表 5 前 3 h 内药材不同位置的温度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.1899	32.3828	32.9667	33.9614	35.4135
1.0	40.3815	40.5535	41.0605	41.8783	42.9977
1.5	45.8464	45.9344	46.1926	46.6048	47.1398
2.0	48.4499	48.4878	48.5982	48.7733	49.0031
2.5	49.4668	49.4790	49.5135	49.5653	49.6610
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

以两问均覆盖的 0.5 h 为基准，问题二中心和表面温度分别为 32.1899 $^{\circ}\text{C}$ 和 35.4135 $^{\circ}\text{C}$ ，

比问题一分别低 1.3857°C 和 1.3722°C。这是由于本问将常物性改为 $\rho(C)$ 、 $c_p(C)$ 和 $k(C)$ ，水分变化会同步改变药材的蓄热与导热过程，使初期升温有所延迟。随着加热延长至 3 h，中心和表面温度升至 49.8495°C 和 49.9664°C，二者仅相差 0.1169°C，温度场已基本均匀。水分计算结果如下。

表 6 前 3 h 内药材不同位置的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5255	2.3257	1.6486
1.0	2.5257	2.4947	2.3578	2.0230	1.4711
1.5	2.3860	2.3256	2.1344	1.8020	1.3476
2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	1.9567	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7166	1.5702	1.3333	1.0081

水分场的变化更能体现温湿耦合的影响。在 0.5 h 时，问题二表面水分 1.6486 kg/kg，高于问题一的 1.5102 kg/kg，而 $r = 1.5\text{ cm}$ 处由 2.3754 kg/kg 降至 2.3257 kg/kg，说明 $D(C, T)$ 随状态更新后，水分分布更加平缓，干燥影响更早向内部传递。延长至 3 h 后，中心、表面和截面平均水分分别降至 1.7662、1.0081 和 1.3825 kg/kg：相比问题一结束时中心水分几乎未变，失水区域此时已扩展至整个截面；但中心与表面仍相差 0.7581 kg/kg，表明后续干燥仍主要受内部水分扩散限制。

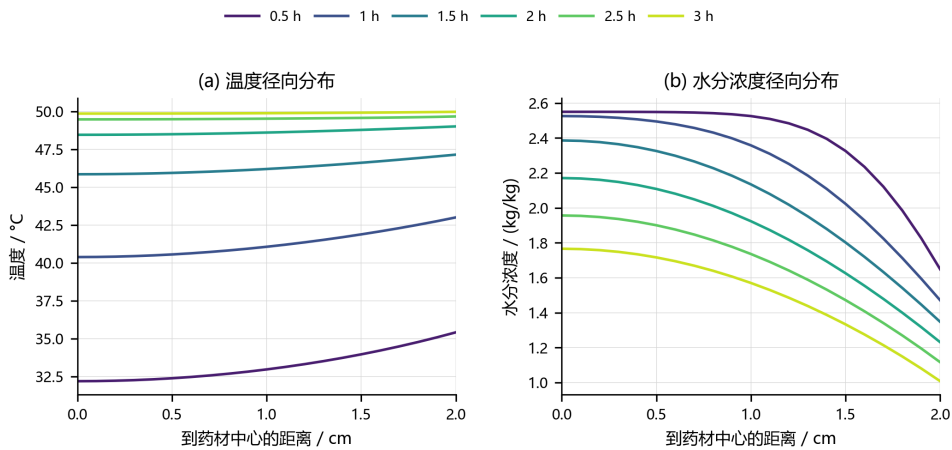


图 8 前 3 h 内药材温度与水分浓度的径向分布

5.4 问题三模型的建立与求解

5.4.1 长期环境边界的构造

附件 1 只给出 0 ~ 4 h 的烘房温度和环境含水率。由于 3 ~ 4 h 内两者已在稳定水平附近小幅波动，主模型以该时段的时间加权均值预测 4 ~ 72 h 的环境边界。对

$y_e(t) \in \{T_e(t), C_e(t)\}$, 取

$$\bar{y}_e = \frac{1}{3600} \int_{10800}^{14400} y_e(t) dt \approx \frac{1}{3600} \sum_j \frac{y_{e,j} + y_{e,j+1}}{2} (t_{j+1} - t_j). \quad (14)$$

计算得到

$$\bar{T}_e = 49.9959^\circ\text{C}, \quad \bar{C}_e = 0.049990 \text{ kg/kg}.$$

因此, 完整环境边界写为

$$(T_0(t), C_0(t)) = \begin{cases} (T_e(t), C_e(t)), & 0 \leq t \leq 4 \text{ h}, \\ (\bar{T}_e, \bar{C}_e), & 4 < t \leq 72 \text{ h}, \end{cases} \quad (15)$$

加权均值算法下问题三可得到唯一的基准干燥时间, 但考虑到实际烘房在接下来的时间内温度和水分浓度仍会波动, 后文在模型检验部分另以 3 ~ 4 h 数据拟合 Huber-OU 随机过程并模拟 4 ~ 72 h 参数波动环境, 用于检验该基准结果的稳健性。

5.4.2 热—湿耦合模型及其求解

第三问在不考虑收缩和蒸发潜热的条件下, 完全沿用问题二的热湿耦合控制方程, 详见表4中的物性关系以及初边值条件。本题相较于问题二区别仅在于将环境边界继续做时间延长, 并从 $t = 0$ 重新推进温度场和水分场, 直至首次满足全域干燥条件, 而不是直接续接问题二的若干输出值。

数值求解沿用问题二的方法, 仍采用 160 个表面加密径向控制体、后向 Euler 格式、分块 Picard 迭代和 Thomas 追赶法, 但根据题目要求每 60s 保存一次径向结果, 详细求解结果见支撑材料的“result3.xlsx”。

5.4.3 干燥终点判定与计算结果

题目要求药材内部所有位置的干基含水率均严格低于 $C_d = 0.15 \text{ kg/kg}$, 因此不能用表面值或截面平均值代替终点判据。每个时间层均比较重构中心值、全部控制体中心值和重构表面值, 定义

$$C_{\max}(t) = \max\{C(0, t), C_1(t), \dots, C_N(t), C_s(t)\}, t_d = \inf\{t : C_{\max}(t) < C_d\} \quad (16)$$

为保证长期计算效率和终点精度, 本题前 4h 采用 1s 步长跟踪实测边界, 4h 后改用 30s; 当 $C_{\max} < 0.152 \text{ kg/kg}$ 时恢复为 1s, 以定位首次越过阈值的时刻。由于题目结果限制四位小数, 因此求解过程保证各处水分浓度严格小于 C_d , 而不是化简后满足终止条件, 计算得到 $t_d = 207017 \text{ s} = 57.5047 \text{ h}$, 终点控制位置为圆柱中心 $r = 0$ 。不同时刻

的径向含水率如下。

表 7 恒均值模型不同时间和径向位置的干基含水率 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
6	1.0174	0.9885	0.9019	0.7553	0.5330
12	0.4565	0.4435	0.4034	0.3300	0.1644
18	0.2991	0.2918	0.2689	0.2254	0.0845
24	0.2379	0.2329	0.2168	0.1856	0.0665
30	0.2059	0.2019	0.1892	0.1641	0.0598
36	0.1859	0.1826	0.1719	0.1504	0.0566
42	0.1721	0.1692	0.1598	0.1407	0.0548
48	0.1619	0.1592	0.1507	0.1335	0.0537
54	0.1539	0.1515	0.1437	0.1277	0.0530
烘干结束时间	0.1500	0.1477	0.1402	0.1249	0.0526

药材表面约在 12.57 h 达标，而中心直到 57.50 h 才越过阈值，说明完全干燥时间由最湿的中心位置决定。相比之下，药材内部最大温差在约 2.72 h 后已低于 0.1℃，因此长期阶段的限制环节是内部水分扩散而非升温过程。

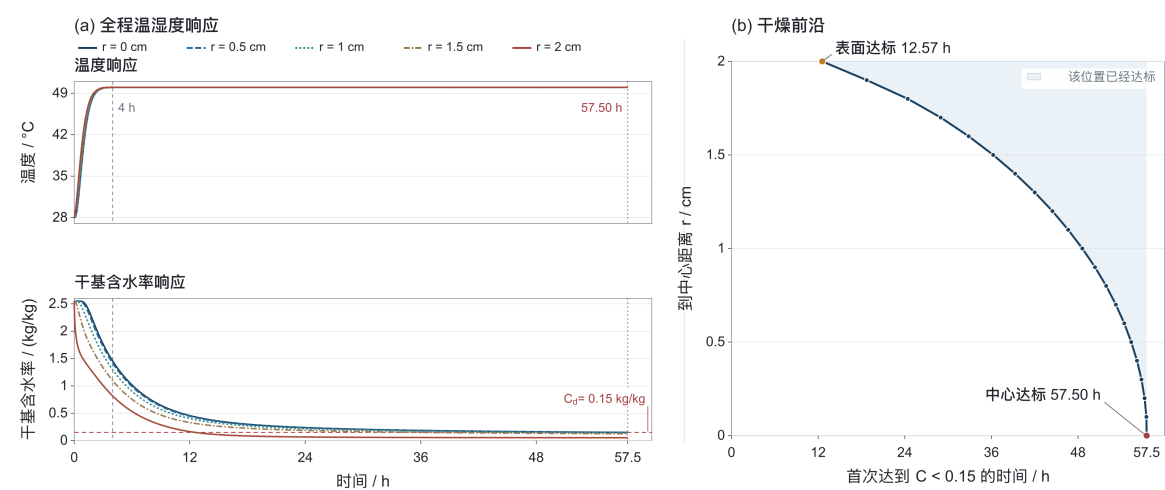


图 9 药材全程温湿度响应与干燥前沿

5.4.4 模型检验与环境扰动分析

为检验 4h 后边界取自实测末段 1h 平均值的合理性以及该处理对结果的影响，我们对平均温度、环境水分及平均时间窗分别进行灵敏度分析扰动。

通过归一化灵敏度对比，得到结果：4 h 后边界取平均值总体稳健，其中最终结果对温度灵敏度约为水分灵敏度的 90 倍，时间窗选择不构成实质性偏差。说明我们对最后 1 h 取平均值作为延拓条件的假设合理。

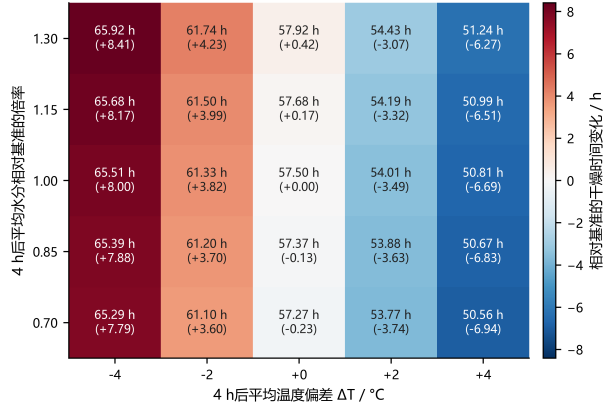


图 10 温度——水分联合扰动对干燥时间的影响

恒均值边界代表典型稳定工况，但实际烘房仍存在温控误差和气流扰动。为了判断这类波动是否会改变干燥终点，我们另用 3 ~ 4 h 的 61 个样本拟合 **Huber-OU 随机边界**并引入 OU 过程。

结果表明引入 OU 随机边界对第三问烘干时间的影响极小：均值偏移 -0.27 min (-0.008%)，单条路径最大偏差仅 1.07 min，96.7% 的蒙特卡洛路径落在基线 ± 1 min 以内。原模型（4 h 后恒值边界）的结果 57.50 h 对 4 h 后边界处理方式是高度稳健的。**说明我们对最后 1 h 取平均值作为延拓条件的假设合理**。因此，第四问不再详细介绍 OU 过程对原耦合模型求解干燥时间的影响。

5.4.5 模型扩展

以下两项均以第三问的一维径向模型为基准，属于相互独立的扩展：二维模型只增加轴向传递，蒸发潜热模型则保持一维几何并修正表面能量平衡，二者未在同一算例中叠加。

(1) 考虑上下表面二维水热交换 为检验忽略轴向传递的合理性，将原一维径向模型扩展为有限长圆柱的二维轴对称模型。利用圆柱两端的对称性，计算区域取 $0 \leq r \leq 0.02 \text{ m}, 0 \leq z \leq 0.125 \text{ m}$ 。

令 $T = T(r, z, t)$ 、 $C = C(r, z, t)$ ，并沿用问题二的状态相关物性，则二维轴对称热、水分方程分别为

$$\begin{cases} \rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rk(C)\frac{\partial T}{\partial r}\right] + \frac{\partial}{\partial z}\left[k(C)\frac{\partial T}{\partial z}\right], \\ \frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rD(C,T)\frac{\partial C}{\partial r}\right] + \frac{\partial}{\partial z}\left[D(C,T)\frac{\partial C}{\partial z}\right]. \end{cases} \quad (17)$$

轴线 $r = 0$ 与中截面 $z = 0$ 均满足对称条件；侧面 $r = R$ 和端面 $z = L/2$ 采用相同的对

流边界：

$$-k_s \frac{\partial T}{\partial n} = h_T [T_s - T_0(t)], -D_s \frac{\partial C}{\partial n} = h_m [C_s - C_0(t)], \quad (18)$$

其中 n 为外法向, $k_s = k(C_s)$, $D_s = D(C_s, T_s)$ 。初始条件、环境边界及干燥判据均与第三问基准模型一致。

结果表明：原一维模型结果为 57.5047 h，考虑端面效应后结果为 57.5025 h，对结果影响仅有 0.0039%。两端面贡献了约 5.30% 的累计排湿量，但主要作用在端部局部区域。圆柱中段仍与原一维模型非常接近。

(2) 考虑蒸发潜热影响 原模型仅考虑药材内部显热变化，未计入表面水分蒸发所消耗的潜热。为评价该简化对计算结果的影响，在保持几何尺寸、物性参数、环境边界、传质系数及干燥判据不变的条件下，对表面能量边界进行修正。

潜热模型保持一维径向几何。由于 C 是干基含水率，单位体积内的水质量应写为 $\rho_d C$ ，其中 $\rho_d = m_d/V$ 为干物质体积密度。因此，加入潜热前的内部守恒方程应表示为

$$\begin{cases} \rho(C)c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[rk(C) \frac{\partial T}{\partial r} \right], \\ \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[rD(C, T) \frac{\partial C}{\partial r} \right]. \end{cases} \quad (19)$$

第三问中圆柱体积和干物质质量均保持不变，故

$$\rho_d = \frac{\rho(C_{\text{ini}})}{1 + C_{\text{ini}}} = 275.04 \text{ kg/m}^3, \quad C_{\text{ini}} = 2.55 \text{ kg/kg}$$

为常数。此时内部水分方程两侧的 ρ_d 可以约去，但在表面必须乘回 ρ_d ，把干基含水率通量转换为实际蒸发水质量通量：

$$j_w = -\rho_d D_s \frac{\partial C}{\partial r} \Big|_{r=R} = \rho_d h_m [C_s - C_0(t)], \quad (20)$$

其中 $j_w > 0$ 表示水分由药材向外蒸发，单位为 $\text{kg}/(\text{m}^2 \cdot \text{s})$ 。取

$$L_v(T_s) = [2500.8 - 2.36T_s] \times 10^3 \text{ J/kg} \quad (21)$$

则蒸发潜热通量为 $q_{\text{lat}} = L_v(T_s)j_w$ ，表面能量平衡修正为

$$h_T [T_0(t) - T_s] = k_s \frac{\partial T}{\partial r} \Big|_{r=R} + L_v(T_s)j_w. \quad (22)$$

比较核心模型与引入蒸发潜热的扩展模型，由图11潜热的影响主要集中在干燥前期。计入潜热后，药材表面最低温度降至 10.23 °C，与常规模型相比温差迅速扩大，最

大降温达到 24.47 °C。

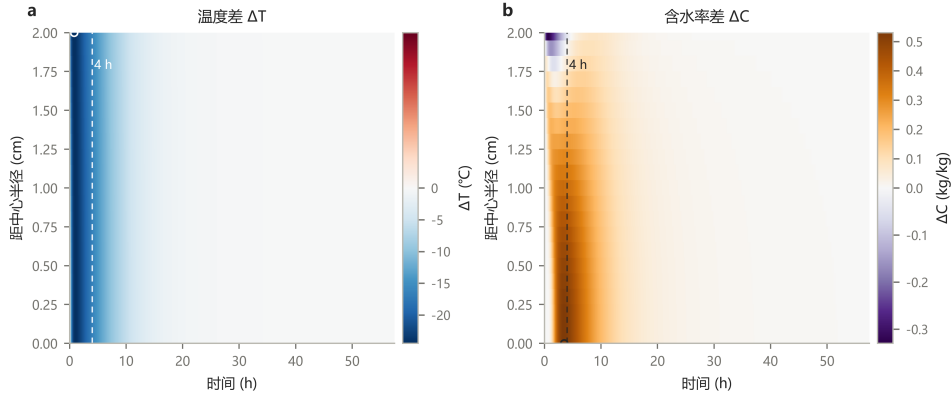


图 11 蒸发潜热相对常规模型的水热影响

低温使扩散系数 $D(C, T)$ 减小，从而抑制内部水分向表面迁移。6 h 时，潜热模型的中心水分浓度为 1.4253 kg/kg，比常规模型高 0.4079 kg/kg，说明内部出现了明显的水分滞留。全过程能量统计表明，进入药材表面的对流热量中约 97.53% 用于水分蒸发，仅 2.47% 转化为药材显热。随着表面逐渐变干，蒸发通量及潜热负荷不断降低，两种模型的温度差逐渐减小。计入蒸发潜热后，药材烘干时间由 57.5047 h 增加至 60.5506 h，延长 3.0458 h，相对增加约 5.30%

5.5 问题四模型的建立与求解

问题四在问题三热-湿耦合框架上加入径向均匀收缩。除按附录 4 更新物性关系外，初始条件、环境边界和干燥判据均沿用问题三，以下只给出移动区域及其引起的方程和离散修正。

5.5.1 时变半径与归一化控制方程

设附件 2 在 t_j 时刻给出的半径为 R_j ，相邻观测点间采用分段线性插值：

$$R(t) = R_j + \frac{R_{j+1} - R_j}{t_{j+1} - t_j}(t - t_j), \quad t_j \leq t \leq t_{j+1}. \quad (23)$$

于是物理区域由问题三的固定区间变为 $0 \leq r \leq R(t)$ 。为在固定区域上计算，引入归一化径向坐标

$$\xi = \frac{r}{R(t)}, \quad \hat{T}(\xi, t) = T(\xi R(t), t), \quad \hat{C}(\xi, t) = C(\xi R(t), t), \quad 0 \leq \xi \leq 1 \quad (24)$$

均匀径向收缩时，材料速度为 $u_r = \dot{R}(t)r/R(t)$ ，故对任意状态量 ϕ 有

$$\frac{D\phi}{Dt} = \frac{\partial\phi}{\partial t}\bigg|_r + u_r \frac{\partial\phi}{\partial r} = \frac{\partial\hat{\phi}}{\partial t}\bigg|_\xi \quad (25)$$

因此固定 ξ 的时间导数已经随材料运动，无须再重复加入网格对流项。定义归一化扩散算子

$$\mathcal{L}_{\Gamma, R(t)}[\hat{\phi}] \equiv \frac{1}{R^2(t)\xi} \frac{\partial}{\partial \xi} \left(\xi \Gamma \frac{\partial \hat{\phi}}{\partial \xi} \right) \quad (26)$$

则问题三的控制方程更新为

$$\begin{cases} \rho(\hat{C})c_p(\hat{C}) \frac{\partial \hat{T}}{\partial t} = \mathcal{L}_{k(\hat{C}), R(t)}[\hat{T}], \\ \frac{\partial \hat{C}}{\partial t} = \mathcal{L}_{D(\hat{C}, \hat{T}), R(t)}[\hat{C}]. \end{cases} \quad (27)$$

其中 $R^{-2}(t)$ 表示收缩对内部传递尺度的修正。物性参数按附录 4 取

$$\begin{cases} \rho(C) = 760 + 90C, \\ c_p(C) = 1850 + 2150 \frac{C}{C+1}, \\ k(C) = 0.12 + 0.20 \frac{C}{C+1}, \\ D(C, T) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T}\right). \end{cases} \quad (28)$$

式中扩散系数的温度采用 K。中心对称条件不变；由于 $\partial/\partial r = R^{-1}(t)\partial/\partial \xi$ ，移动表面 $\xi = 1$ 的边界更新为

$$\begin{aligned} \xi = 0 : \quad & \frac{\partial \hat{T}}{\partial \xi}\bigg|_{\xi=0} = 0, \quad \frac{\partial \hat{C}}{\partial \xi}\bigg|_{\xi=0} = 0, \\ \xi = 1 : \quad & \frac{k_s}{R(t)} \frac{\partial \hat{T}}{\partial \xi}\bigg|_{\xi=1} = h_T [T_0(t) - \hat{T}_s], \\ & -\frac{D_s}{R(t)} \frac{\partial \hat{C}}{\partial \xi}\bigg|_{\xi=1} = h_m [\hat{C}_s - C_0(t)]. \end{aligned} \quad (29)$$

5.5.2 归一化网格上的数值更新

数值格式沿用问题三的后向 Euler–分块 Picard–Thomas 求解框架，只更新与收缩有关的几何量。在 $\xi \in [0, 1]$ 上固定 $N = 320$ 个向表面加密的控制体，其界面取

$$\xi_{f,j} = 1 - \left(1 - \frac{j}{N}\right)^2, \quad j = 0, 1, \dots, N. \tag{30}$$

进入时间层 t_{n+1} 时，先计算 $R^{n+1} = R(t_{n+1})$ ，再映射并更新

$$r_{f,j}^{n+1} = R^{n+1} \xi_{f,j}, \quad r_{f,i}^{n+1} = \pi \left[\left(r_{f,i}^{n+1}\right)^2 - \left(r_{f,i-1}^{n+1}\right)^2 \right], \quad r_{f,j}^{n+1} = 2\pi r_{f,j}^{n+1}. \tag{31}$$

随后用新的 r_f 、 V_i 、 A_f 及表面半单元距离重组问题三的有限体积系数；这等价于在控制方程中逐步更新 $R^{-2}(t)$ ，并在表面边界中更新 $R^{-1}(t)$ 。其余物性迭代、线性方程组求解和终点搜索均沿用问题三，仍以全域最大含水率首次低于 0.15 kg/kg 作为停止条件。

5.5.3 求解结果与分析

根据前述全域最大含水率判据计算各时刻药材内部的最大水分浓度。当其首次低于 0.15 kg/kg 时，判定药材完成烘干。计算得到考虑尺寸变化后的药材烘干时间为

$$t_d = 184022 \text{ s} = 51.1172 \text{ h}$$

终止时最大水分浓度位于药材中心。为展示干燥过程中水分浓度的空间变化，表 8 给出了每隔 6 h 时药材内部有效固定位置及移动表面的计算结果。由于药材在 6 h 时的半径已收缩至 1.374 cm，且此后继续减小，因此表中固定位置仅列出 0、0.5 和 1.0 cm。

表 8 考虑尺寸收缩的药材烘干过程水分浓度

时间/h	到药材中心的距离/cm			
	0	0.5	1.0	药材表面
6	1.7198	1.5380	1.0228	0.4208
12	0.7382	0.6551	0.4086	0.1671
18	0.4091	0.3689	0.2399	0.0892
24	0.2853	0.2612	0.1791	0.0674
30	0.2265	0.2095	0.1493	0.0595
36	0.1929	0.1797	0.1317	0.0560
42	0.1713	0.1604	0.1200	0.0541
48	0.1562	0.1469	0.1116	0.0530
烘干结束时间	0.1500	0.1413	0.1081	0.0526

表中水分浓度单位均为 kg/kg，“药材表面”表示各时刻 $r = R(t)$ 处的移动边界值。1.5 cm 和 2.0 cm 处在所有列示时刻均已位于收缩后的药材外部，故不再作为有效计算位

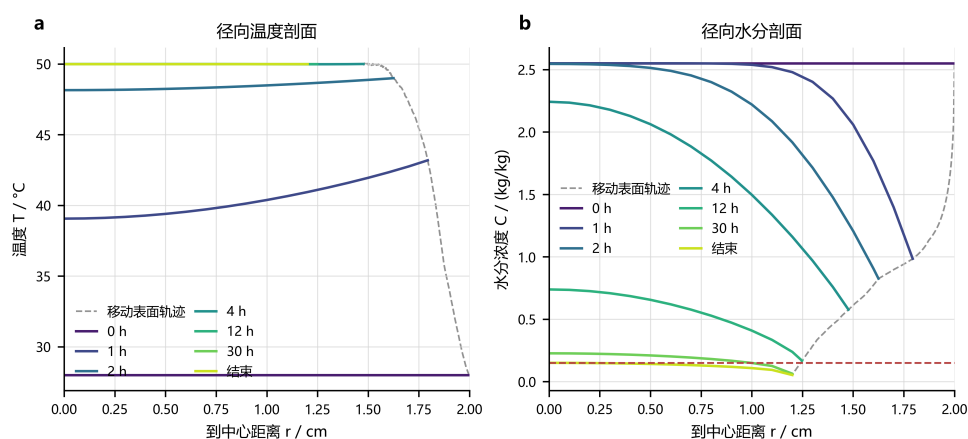


图 12 不同时间水热径向剖面图

置列入表中。完整结果文件仍按照题目要求，以 60 s 为时间间隔、0.1 cm 为距离间隔保存，位于当前药材外部的的位置留空。

由表 8 可见，药材表面的水分浓度下降较快，而中心位置下降较慢，说明干燥后期主要受内部水分迁移过程控制。烘干结束时，中心和表面的水分浓度分别为 0.1500 和 0.0526 kg/kg。其中中心值 0.1500 是保留四位小数后的显示结果，程序用于判断终点的未舍入值为

$$C_{\max}(184022 \text{ s}) = 0.1499998440 \text{ kg/kg} < 0.15 \text{ kg/kg}.$$

因此，终止时药材内部所有位置的水分浓度均满足题目规定的烘干要求。

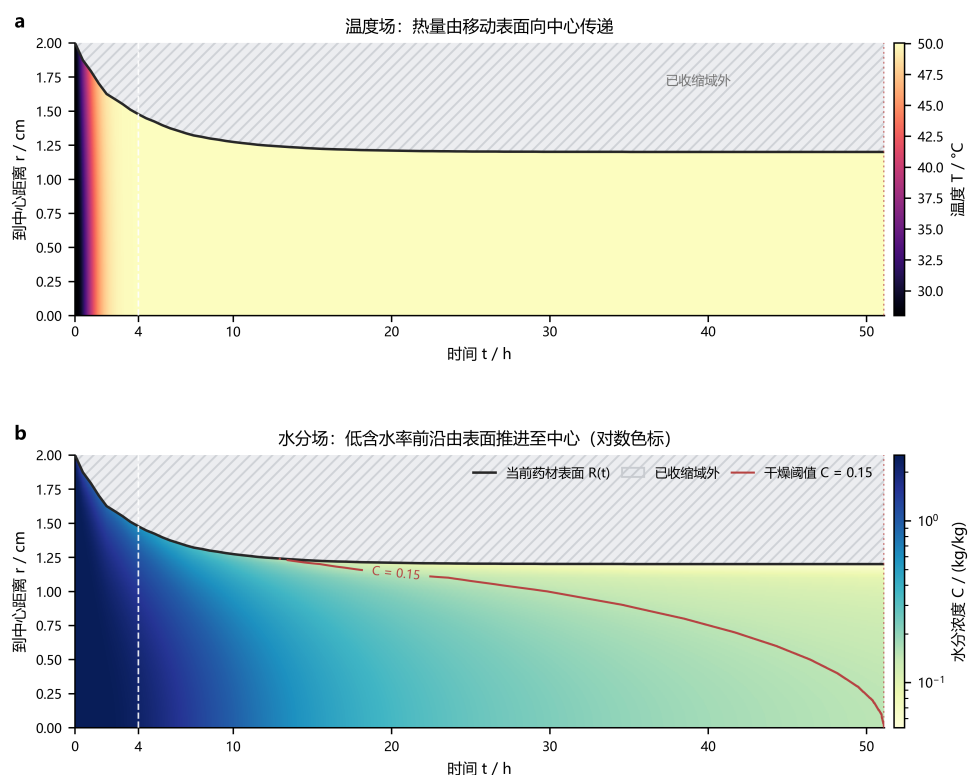


图 13 收缩药材温度与水分浓度的时空演化图

图13黑色曲线表示移动表面 $R(t)$ ，灰色斜线区域表示药材外部，水分图中的红色曲线为 $C = 0.15 \text{ kg/kg}$ 的达标前沿。温度场在约 4 小时后已基本达到稳定，而水分场仍存在明显的中心—表面梯度，说明烘干后期主要受内部水分迁移控制。移动表面约在 12.8 h 达到干燥阈值，但药材中心直至 51.1172 h 才达标。因此，若仅依据表面状态判断烘干结束，将明显低估所需时间。药材半径在 35 小时达到 1.200 cm 后保持稳定，但此时中心水分浓度仍为 0.1975 kg/kg，还需继续干燥约 16.12 h。这说明尺寸稳定同样不能代替全域水分浓度判据。问题三固定尺寸模型的烘干时间为 57.5047 h，本问结果缩短

$$57.5047 - 51.1172 = 6.3875 \text{ h},$$

相对减少约 11.11%。半径减小缩短了内部迁移距离，是烘干时间下降的重要原因。但本问同时采用了附录 4 中的新物性关系，因此该差值表示尺寸变化与物性变化的综合影响，不能全部解释为收缩的单独贡献。

5.5.4 模型扩展

(1) 蒸发潜热 由问题三的潜热模型扩展得到

表 9 蒸发潜热对烘干时长的影响

模型情景	烘干时间/h	增加时间/h	相对变化/%
常规模型（不计蒸发潜热）	51.1172	—	—
扩展模型（计入表面蒸发潜热）	56.4208	5.3036	10.3754

扩展模型所得烘干时间为 56.4208 h，较常规模型增加约 5.30 h。原因在于表面蒸发消耗部分输入热量，使药材温度降低；而扩散系数满足

$$\frac{\partial \ln D}{\partial T} = \frac{3850}{T^2} > 0,$$

因此温度下降会减小水分扩散系数，延缓内部水分向表面的迁移。说明蒸发耗热可能通过降低温度、减弱水分扩散而推迟达标，其定量影响仍需实验校准。

(2) 二维模型 在问题三蒸发潜热模型的基础上，进一步引入问题四的动态收缩边界，得到考虑表面蒸发潜热的扩展模型。计算结果如表 10 所示。

表 10 考虑端面对烘干时长的影响

模型情景	烘干时间/h	增加时间/s	相对变化/%
常规模型	51.1172	—	—
扩展模型	51.1256	30	0.0163%

结果同第三问，考虑端面效应对干燥终止时间的影响十分微小。

(3) 假设半径不变结果 恒定半径 $R = 2\text{ cm}$ 套用问题三模型，烘干时间为 129.8869 h；考虑附件 2 中的半径收缩后，烘干时间为 51.1172 h；半径收缩会使预测烘干时间缩短约 78.77 h，缩短比例为 60.64%

产生这一差异的主要原因是：药材半径缩小后，中心到表面的最大水分扩散距离明显减小，而扩散特征时间近似与半径平方成正比，即

$$t_{\text{diff}} \propto \frac{R^2}{D}.$$

半径由 2.0 cm 减小到 1.2 cm 后，特征扩散距离平方降为初始值的

$$\left(\frac{1.2}{2.0}\right)^2 = 0.36.$$

因此，内部水分到达表面的扩散阻力大幅降低，中心水分浓度下降速度明显加快。收缩还会改变药材的表面积—体积关系，使单位体积对应的有效传质面积增大，进一步促进水分排出。

六、模型分析与评价

6.1 模型的优点

1. 本文从药材烘干过程中的传热与水分迁移机理出发建立模型，并随着问题的深入逐步考虑物性变化和药材收缩等因素。各问之间层层递进、衔接自然，模型结构清晰，具有较明确的物理意义。

2. 模型综合考虑了温度场和水分场之间的相互影响，能够较为全面地描述药材内部温度与水分随时间和位置的变化，相比只研究单一因素的模型更贴近实际烘干过程。

3. 采用有限体积法对模型进行离散求解，在保证模型合理性的同时降低了计算难度，具有较好的稳定性和可操作性。

4. 模型中各参数含义明确，计算结果直观，既可以判断药材的烘干状态，也可以为类似食品的传热传质及烘干时间预测问题提供一定的参考。

6.2 模型的缺点

1. 为了便于建模，本文将药材视为均匀、各向同性的规则物体，未进一步考虑内部孔隙分布和局部变形的差异，因此模型与实际情况之间仍可能存在一定偏差。

2. 模型中的部分物性参数和边界条件根据已有数据及经验关系确定。当药材配方、

尺寸或烘干设备发生较大变化时，相关参数需要重新测定和调整，模型的直接适用范围因此受到一定限制。

6.3 模型的改进

1. 进一步完善实验数据。后续可以增加不同烘干阶段下药材表面温度、中心温度、整体质量及含水率等数据的测量，并利用实验结果对扩散系数、传热系数和传质系数等参数进行修正，从而提高模型参数的可靠性。

2. 适当细化模型假设。在现有模型的基础上，可以进一步考虑药材内部孔隙分布、局部物性差异等因素，使模型能够更加细致地反映实际烘干过程中温度和水分的变化。

3. 扩大模型的适用范围。后续可以选取不同尺寸的药材以及不同烘干环境进行实验，研究模型参数随原料和工况的变化规律。在主要研究整体烘干时间时，可继续采用计算效率较高的一维模型；当需要分析局部烘干状态时，则可使用二维或三维模型，从而兼顾计算效率与模型精度。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试、语言润色、理论公式讲解，详细使用情况见支撑材料。

参考文献

- [1] 王乐意, 李长河, 刘明政, 李康, 叶祥瑞, 王子文, 范林政, 王仁坤, 赵华洋, 坎杂, et al. 中药材干燥技术与装备研究现状. Transactions of the Chinese Society of Agricultural Engineering, 40(2):1, 2024.
- [2] Qian Liu, Meng Li, Heng Lu, Yongli Zhou, and Xiao Wang. Effect of steam pretreatment on drying kinetics and quality of polygonati rhizoma dried in hot-blast air. Journal of Food Process Engineering, 45(1):e13925, 2022.
- [3] Hong Thai Vu and Evangelos Tsotsas. Mass and heat transport models for analysis of the drying process in porous media: a review and numerical implementation. International Journal of Chemical Engineering, 2018(1):9456418, 2018.

附录 A 模板所用的宏包

表 11 宏包罗列

模板中已经加载的宏包				
amsbsy	amsfonts	amsgen	amsmath	amsopn
amssymb	amstext	appendix	array	atbegshi
atveryend	auxhook	bigdelim	bigintcalc	bigstrut
bitset	bm	booktabs	calc	caption
caption3	CJKfntef	cprotect	ctex	ctexhook
ctexpatch	enumitem	etexcmds	etoolbox	everysel
expl3	fix-cm	fontenc	fontspec	fontspec-xetex
geometry	getttitlestring	graphics	graphicx	hobsub
hobsub-generic	hobsub-hyperref	hopatch	hxetex	hycolor
hyperref	ifluatex	ifpdf	ifthen	ifvtex
ifxetex	indentfirst	infwarerr	intcalc	keyval
kvdefinekeys	kvoptions	kvsetkeys	l3keys2e	letltxmacro
listings	longtable	lstmisc	ltxcaption	ltxcmds
multirow	nameref	pdfescape	pdftexcmds	refcount
rerunfilecheck	stringenc	suffix	titletoc	tocloft
trig	ulem	uniquecounter	url	xcolor
xcolor-patch	xeCJK	xeCJKfntef	xeCJK-listings	xparse
xtemplate	zhnumber			

附录 B 问题一代码

```
"""A problem, question 1: temperature and moisture fields by finite volumes.
```

```
The herb is approximated as a long axisymmetric cylinder. Under Appendix 2,
the heat and moisture equations share the same geometry, time grid, and
measured oven environment, but the supplied constitutive laws contain no
temperature-moisture cross term. Therefore this script performs a synchronized
co-simulation without inventing an unprovided latent-heat parameter.
```

```
Outputs are restricted to CSV/JSON data and PNG figures as requested.
```

```
"""
```

```
from __future__ import annotations
```

```

import argparse
import csv
import json
import math
from dataclasses import dataclass
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import numpy as np
from openpyxl import load_workbook

import a1_temperature_fvm as heat

RADIUS_M = 0.02
INITIAL_MOISTURE_KG_KG = 2.55
MASS_TRANSFER_COEFFICIENT_M_S = 8.0e-7
END_TIME_S = 1800.0

TABLE_TIMES_S = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=float)
OUTPUT_RADII_CM = np.round(np.arange(0.0, 2.0 + 0.05, 0.1), 1)

@dataclass
class EnvironmentData:
    times_s: np.ndarray
    temperature_c: np.ndarray
    moisture_kg_kg: np.ndarray

@dataclass
class MoistureResult:
    times_s: np.ndarray
    radii_m: np.ndarray
    moisture_kg_kg: np.ndarray
    surface_moisture_kg_kg: np.ndarray
    volumes_m3_m: np.ndarray
    boundary_outflow_m3_kg_kg_m: float
    maximum_picard_iterations: int

```

```

def configure_fonts() -> None:
    """Choose a local CJK-capable font and competition-paper styling."""
    available = {font.name for font in fm.fontManager.ttflist}
    candidates = ["Microsoft YaHei", "SimHei", "Arial", "DejaVu Sans"]
    selected = [name for name in candidates if name in available]
    plt.rcParams.update(
        {
            "font.family": "sans-serif",
            "font.sans-serif": selected or ["DejaVu Sans"],
            "axes.unicode_minus": False,
            "svg.fonttype": "none",
            "pdf.fonttype": 42,
            "font.size": 8.0,
            "axes.labelsize": 8.0,
            "axes.titlesize": 9.0,
            "xtick.labelsize": 7.0,
            "ytick.labelsize": 7.0,
            "legend.fontsize": 7.0,
            "axes.linewidth": 0.7,
        }
    )

def find_default_data_path(repo_root: Path) -> Path:
    path = (
        repo_root
        / "比赛题目"
        / "CUMCM2026Problems"
        / "A题"
        / "附件"
        / "附件1.xlsx"
    )
    if not path.exists():
        raise FileNotFoundError(f"Cannot find Attachment 1: {path}")
    return path

def load_environment(path: Path) -> EnvironmentData:
    """Read time, oven temperature, and oven moisture without editing the file."""
    workbook = load_workbook(path, read_only=True, data_only=True)
    sheet = workbook.active
    rows = list(sheet.iter_rows(min_row=2, values_only=True))
    workbook.close()

    data = [row[:3] for row in rows if row[0] is not None]
    times = np.asarray([float(row[0]) for row in data], dtype=float)
    temperatures = np.asarray([float(row[1]) for row in data], dtype=float)

```

```

moistures = np.asarray([float(row[2]) for row in data], dtype=float)

if not np.all(np.diff(times) > 0.0):
    raise ValueError("Attachment 1 time values must be strictly increasing.")
if times[0] > 0.0 or times[-1] < END_TIME_S:
    raise ValueError("Attachment 1 does not cover the required 0-1800 s interval.")
if not (np.all(np.isfinite(temperatures)) and np.all(np.isfinite(moistures))):
    raise ValueError("Attachment 1 contains a non-finite environmental value.")

return EnvironmentData(times, temperatures, moistures)

def moisture_diffusivity(moisture_kg_kg: np.ndarray) -> np.ndarray:
    """Appendix 2 empirical diffusivity  $D=7e-9 \exp(-0.89/C)$ , in  $m^2/s$ ."""
    moisture = np.asarray(moisture_kg_kg, dtype=float)
    if np.any(moisture <= 0.0):
        raise ValueError("Moisture concentration must remain positive in D(C).")
    return 7.0e-9 * np.exp(-0.89 / moisture)

def factor_tridiagonal(
    lower: np.ndarray, diagonal: np.ndarray, upper: np.ndarray
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Factor a tridiagonal matrix for Thomas solves."""
    n = diagonal.size
    denominators = np.empty(n, dtype=float)
    upper_prime = np.empty(n - 1, dtype=float)
    denominators[0] = diagonal[0]
    if denominators[0] <= 0.0:
        raise ValueError("Invalid tridiagonal system.")
    upper_prime[0] = upper[0] / denominators[0]
    for i in range(1, n - 1):
        denominators[i] = diagonal[i] - lower[i - 1] * upper_prime[i - 1]
        if denominators[i] <= 0.0:
            raise ValueError("Invalid tridiagonal system.")
        upper_prime[i] = upper[i] / denominators[i]
    denominators[-1] = diagonal[-1] - lower[-1] * upper_prime[-1]
    if denominators[-1] <= 0.0:
        raise ValueError("Invalid tridiagonal system.")
    return lower.copy(), upper_prime, denominators

def solve_factored_tridiagonal(
    factor: tuple[np.ndarray, np.ndarray, np.ndarray], rhs: np.ndarray
) -> np.ndarray:
    """Solve a factored tridiagonal linear system."""
    lower, upper_prime, denominators = factor

```

```

n = rhs.size
rhs_prime = np.empty(n, dtype=float)
rhs_prime[0] = rhs[0] / denominators[0]
for i in range(1, n):
    rhs_prime[i] = (
        rhs[i] - lower[i - 1] * rhs_prime[i - 1]
    ) / denominators[i]
solution = np.empty(n, dtype=float)
solution[-1] = rhs_prime[-1]
for i in range(n - 2, -1, -1):
    solution[i] = rhs_prime[i] - upper_prime[i] * solution[i + 1]
return solution

def build_moisture_system(
    radii_m: np.ndarray,
    west_faces_m: np.ndarray,
    east_faces_m: np.ndarray,
    volumes_m3_m: np.ndarray,
    moisture_iterate: np.ndarray,
    time_step_s: float,
) -> tuple[tuple[np.ndarray, np.ndarray, np.ndarray], float, float]:
    """Build one Picard-linearized backward-Euler moisture system."""
    n = radii_m.size
    diffusivity_nodes = moisture_diffusivity(moisture_iterate)
    lower = np.zeros(n - 1, dtype=float)
    diagonal = np.ones(n, dtype=float)
    upper = np.zeros(n - 1, dtype=float)

    for i in range(n):
        if i > 0:
            west_resistance = (
                (west_faces_m[i] - radii_m[i - 1]) / diffusivity_nodes[i - 1]
                + (radii_m[i] - west_faces_m[i]) / diffusivity_nodes[i]
            )
            conductance_w = 2.0 * math.pi * west_faces_m[i] / west_resistance
            coefficient_w = conductance_w / volumes_m3_m[i]
            diagonal[i] += time_step_s * coefficient_w
            lower[i - 1] = -time_step_s * coefficient_w
        if i < n - 1:
            east_resistance = (
                (east_faces_m[i] - radii_m[i]) / diffusivity_nodes[i]
                + (radii_m[i + 1] - east_faces_m[i]) / diffusivity_nodes[i + 1]
            )
            conductance_e = 2.0 * math.pi * east_faces_m[i] / east_resistance
            coefficient_e = conductance_e / volumes_m3_m[i]
            diagonal[i] += time_step_s * coefficient_e

```

```

        upper[i] = -time_step_s * coefficient_e

    surface_area_per_length = 2.0 * math.pi * RADIUS_M
    # The outer unknown is at the centre of the last control volume. The
    # half-cell diffusion resistance and air-side convection resistance are
    # placed in series, avoiding a first-order surface-node approximation.
    boundary_conductance = surface_area_per_length / (
        (RADIUS_M - radii_m[-1]) / diffusivity_nodes[-1]
        + 1.0 / MASS_TRANSFER_COEFFICIENT_M_S
    )
    boundary_coefficient = boundary_conductance / volumes_m3_m[-1]
    diagonal[-1] += time_step_s * boundary_coefficient
    return (
        factor_tridiagonal(lower, diagonal, upper),
        boundary_coefficient,
        boundary_conductance,
    )

def simulate_moisture(
    source_times_s: np.ndarray,
    source_moisture_kg_kg: np.ndarray,
    radial_step_cm: float,
    time_step_s: float,
    picard_tolerance: float = 1.0e-10,
    maximum_picard_iterations: int = 50,
) -> MoistureResult:
    """Solve nonlinear radial moisture diffusion with a convective boundary."""
    radial_step_m = radial_step_cm / 100.0
    intervals = int(round(RADIUS_M / radial_step_m))
    steps = int(round(END_TIME_S / time_step_s))
    if not math.isclose(intervals * radial_step_m, RADIUS_M, abs_tol=1e-12):
        raise ValueError("The radial step must divide the 0.02 m radius exactly.")
    if not math.isclose(steps * time_step_s, END_TIME_S, abs_tol=1e-12):
        raise ValueError("The time step must divide 1800 s exactly.")
    record_stride = int(round(1.0 / time_step_s))
    if not math.isclose(record_stride * time_step_s, 1.0, abs_tol=1e-12):
        raise ValueError("The time step must divide the 1 s output interval exactly.")

    # A quadratic face mapping clusters cells near the drying surface, where an
    # initially very thin moisture boundary layer forms. The interior remains
    # coarser because it is nearly uniform during the first 30 minutes.
    logical_faces = np.linspace(0.0, 1.0, intervals + 1)
    all_faces = RADIUS_M * (1.0 - (1.0 - logical_faces) ** 2)
    west_faces = all_faces[:-1]
    east_faces = all_faces[1:]
    radii = 0.5 * (west_faces + east_faces)

```

```

volumes = math.pi * (east_faces**2 - west_faces**2)

output_times = np.arange(0.0, END_TIME_S + 1.0, 1.0)
moisture = np.full(radii.size, INITIAL_MOISTURE_KG_KG, dtype=float)
records = np.empty((output_times.size, radii.size), dtype=float)
records[0] = moisture
surface_records = np.empty(output_times.size, dtype=float)
surface_records[0] = INITIAL_MOISTURE_KG_KG
boundary_outflow = 0.0
surface_area_per_length = 2.0 * math.pi * RADIUS_M
record_index = 1
maximum_iterations_used = 0

for step in range(1, steps + 1):
    time_now = step * time_step_s
    oven_moisture = float(
        np.interp(time_now, source_times_s, source_moisture_kg_kg)
    )
    previous = moisture.copy()
    iterate = previous.copy()

    for iteration in range(1, maximum_picard_iterations + 1):
        factor, boundary_coefficient, boundary_conductance = build_moisture_system(
            radii,
            west_faces,
            east_faces,
            volumes,
            iterate,
            time_step_s,
        )
        rhs = previous.copy()
        rhs[-1] += time_step_s * boundary_coefficient * oven_moisture
        updated = solve_factored_tridiagonal(factor, rhs)
        if np.max(np.abs(updated - iterate)) < picard_tolerance:
            iterate = updated
            break
        iterate = updated
    else:
        raise RuntimeError(f"Moisture Picard iteration failed at t={time_now:g} s.")

maximum_iterations_used = max(maximum_iterations_used, iteration)
moisture = iterate
surface_diffusivity = float(moisture_diffusivity(moisture[-1:])[0])
surface_distance = RADIUS_M - radii[-1]
surface_moisture = (
    (surface_diffusivity / surface_distance) * moisture[-1]
    + MASS_TRANSFER_COEFFICIENT_M_S * oven_moisture

```

```

    ) / (
        surface_diffusivity / surface_distance
        + MASS_TRANSFER_COEFFICIENT_M_S
    )
    boundary_outflow += (
        time_step_s
        * boundary_conductance
        * (moisture[-1] - oven_moisture)
    )

    if step % record_stride == 0:
        records[record_index] = moisture
        surface_records[record_index] = surface_moisture
        record_index += 1

    return MoistureResult(
        times_s=output_times,
        radii_m=radii,
        moisture_kg_kg=records,
        surface_moisture_kg_kg=surface_records,
        volumes_m3_m=volumes,
        boundary_outflow_m3_kg_kg_m=boundary_outflow,
        maximum_picard_iterations=maximum_iterations_used,
    )

def sample_field(
    times_s: np.ndarray,
    radii_m: np.ndarray,
    field: np.ndarray,
    query_times_s: np.ndarray,
    query_radii_cm: np.ndarray,
) -> np.ndarray:
    """Sample a recorded field at exact integer seconds and requested radii."""
    time_indices = np rint(query_times_s).astype(int)
    if not np.allclose(times_s[time_indices], query_times_s):
        raise ValueError("Requested times are not available in the 1 s records.")
    sampled = np.empty((query_times_s.size, query_radii_cm.size), dtype=float)
    query_radii_m = query_radii_cm / 100.0
    for j, time_index in enumerate(time_indices):
        sampled[j] = np.interp(query_radii_m, radii_m, field[time_index])
    return sampled

def sample_moisture_field(
    result: MoistureResult,
    query_times_s: np.ndarray,

```

```

    query_radai_cm: np.ndarray,
) -> np.ndarray:
    """Sample cell-centred moisture, reconstructing centre and surface values."""
    time_indices = np.rint(query_times_s).astype(int)
    if not np.allclose(result.times_s[time_indices], query_times_s):
        raise ValueError("Requested times are not available in the 1 s records.")
    sampled = np.empty((query_times_s.size, query_radai_cm.size), dtype=float)
    extended_radai = np.concatenate(([0.0], result.radai_m, [RADIUS_M]))
    query_radai_m = query_radai_cm / 100.0
    for j, time_index in enumerate(time_indices):
        nodes = result.moisture_kg_kg[time_index]
        # Even symmetry gives C(r)=a+b*r^2+..., hence this second-order centre
        # reconstruction from values at dr/2 and 3dr/2.
        r0_squared = result.radai_m[0] ** 2
        r1_squared = result.radai_m[1] ** 2
        center_value = (
            nodes[0] * r1_squared - nodes[1] * r0_squared
        ) / (r1_squared - r0_squared)
        extended_values = np.concatenate(
            (
                [center_value],
                nodes,
                [result.surface_moisture_kg_kg[time_index]],
            )
        )
        sampled[j] = np.interp(query_radai_m, extended_radai, extended_values)
    return sampled

def write_field_csv(
    path: Path, times_s: np.ndarray, radai_cm: np.ndarray, values: np.ndarray
) -> None:
    """Write the official 1 s by 0.1 cm rectangular result."""
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(["time_s", *[f"r_{radius:.1f}_cm" for radius in radai_cm]])
        for time_s, row in zip(times_s, values):
            writer.writerow([f"{time_s:.0f}", *[f"{value:.4f}" for value in row]])

def write_table_csv(
    path: Path, times_s: np.ndarray, radai_cm: np.ndarray, values: np.ndarray
) -> None:
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(["时间/s", *[f"{radius:g} cm" for radius in radai_cm]])
        for time_s, row in zip(times_s, values):

```

```

        writer.writerow([f"{time_s:.0f}", *[f"{value:.4f}" for value in row]])

def calculate_surface_fluxes(
    environment: EnvironmentData,
    temperature: heat.SimulationResult,
    moisture: MoistureResult,
) -> dict[str, np.ndarray]:
    """Calculate convective boundary fluxes on the common 1 s time grid.

    Heat flux is positive into the herb. Moisture flux is positive out of the
    herb. Because C is a dry-basis mass ratio, the constitutive boundary flux
     $h_m(C_s - C_{inf})$  is also retained before conversion with the supplied density.
    """
    if not np.array_equal(temperature.times_s, moisture.times_s):
        raise ValueError("Temperature and moisture outputs must share a time grid.")

    times_s = temperature.times_s
    oven_temperature_c = np.interp(
        times_s, environment.times_s, environment.temperature_c
    )
    oven_moisture_kg_kg = np.interp(
        times_s, environment.times_s, environment.moisture_kg_kg
    )
    surface_temperature_c = temperature.temperature_c[:, -1]
    surface_moisture_kg_kg = moisture.surface_moisture_kg_kg

    heat_flux_into_herb_w_m2 = heat.CONVECTION_COEFFICIENT_W_M2_K * (
        oven_temperature_c - surface_temperature_c
    )
    moisture_ratio_flux_out_m_s = MASS_TRANSFER_COEFFICIENT_M_S * (
        surface_moisture_kg_kg - oven_moisture_kg_kg
    )
    moisture_mass_flux_out_kg_m2_s = (
        heat.DENSITY_KG_M3 * moisture_ratio_flux_out_m_s
    )

    return {
        "time_s": times_s,
        "oven_temperature_c": oven_temperature_c,
        "surface_temperature_c": surface_temperature_c,
        "heat_flux_into_herb_w_m2": heat_flux_into_herb_w_m2,
        "oven_moisture_kg_kg": oven_moisture_kg_kg,
        "surface_moisture_kg_kg": surface_moisture_kg_kg,
        "moisture_ratio_flux_out_m_s": moisture_ratio_flux_out_m_s,
        "moisture_mass_flux_out_kg_m2_s": moisture_mass_flux_out_kg_m2_s,
    }

```

```

def write_surface_flux_csv(path: Path, fluxes: dict[str, np.ndarray]) -> None:
    """Write one-second surface boundary states and fluxes to CSV."""
    headers = list(fluxes)
    arrays = [fluxes[name] for name in headers]
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(headers)
        for row in zip(*arrays):
            writer.writerow(
                [f"{row[0]:.0f}", *[f"{value:.10g}" for value in row[1:]]]
            )

def render_surface_flux_png(
    path: Path,
    fluxes: dict[str, np.ndarray],
) -> None:
    """Plot heat input and moisture outflow fluxes as complementary panels."""
    configure_fonts()
    time_minutes = fluxes["time_s"] / 60.0
    heat_flux = fluxes["heat_flux_into_herb_w_m2"]
    moisture_flux_g_m2_s = (
        1000.0 * fluxes["moisture_mass_flux_out_kg_m2_s"]
    )

    figure, axes = plt.subplots(
        1, 2, figsize=(7.09, 3.25), constrained_layout=True
    )
    axes[0].plot(time_minutes, heat_flux, color="#D55E00", linewidth=1.5)
    axes[0].fill_between(
        time_minutes, 0.0, heat_flux, color="#E69F00", alpha=0.16, linewidth=0
    )
    axes[0].set(
        title="(a) 进入药材的表面热流密度",
        xlabel="时间 / min",
        ylabel="热流密度 / (W/m²)",
        xlim=(0.0, END_TIME_S / 60.0),
    )

    axes[1].plot(
        time_minutes, moisture_flux_g_m2_s, color="#009E73", linewidth=1.5
    )
    axes[1].fill_between(
        time_minutes,
        0.0,

```

```

        moisture_flux_g_m2_s,
        color="#009E73",
        alpha=0.14,
        linewidth=0,
    )
    axes[1].set(
        title="(b) 离开药材的表面水分通量",
        xlabel="时间 / min",
        ylabel="水分质量通量 / [g/(m² · s)]",
        xlim=(0.0, END_TIME_S / 60.0),
    )
    for axis in axes:
        style_axis(axis)
        axis.axhline(0.0, color="#555555", linewidth=0.65)
        axis.set_xticks(np.arange(0.0, 31.0, 5.0))
    figure.savefig(path, dpi=300, facecolor="white")
    plt.close(figure)

def render_table_png(
    path: Path,
    table_number: int,
    title: str,
    unit_label: str,
    values: np.ndarray,
) -> None:
    """Render a clean PNG version of one paper table."""
    configure_fonts()
    figure, axis = plt.subplots(figsize=(11.7, 5.6))
    axis.axis("off")
    axis.set_title(f"表{table_number} {title}", fontsize=18, fontweight="bold", pad=34)
    axis.text(
        0.5,
        1.01,
        "到药材中心的距离 / cm",
        transform=axis.transAxes,
        ha="center",
        va="bottom",
        fontsize=13,
    )
    cell_text = [[f"{value:.4f}" for value in row] for row in values]
    row_labels = [f"{time_s:.0f}" for time_s in TABLE_TIMES_S]
    column_labels = [f"{radius:g}" for radius in TABLE_RADII_CM]
    table = axis.table(
        cellText=cell_text,
        rowLabels=row_labels,
        colLabels=column_labels,
    )

```

```

        cellLoc="center",
        rowLoc="center",
        bbox=[0.06, 0.04, 0.91, 0.80],
    )
    table.auto_set_font_size(False)
    table.set_fontsize(12)
    for (row, column), cell in table.get_celld().items():
        cell.set_edgecolor("#7A7A7A")
        cell.set_linewidth(0.8)
        if row == 0:
            cell.set_facecolor("#D9E4F0")
            cell.set_text_props(fontweight="bold")
        if column == -1:
            cell.set_facecolor("#F3F4F6")
            cell.set_text_props(fontweight="bold")
    axis.text(
        -0.015,
        0.815,
        "时间 / s",
        transform=axis.transAxes,
        ha="left",
        va="bottom",
        fontsize=12,
    )
    axis.text(
        0.97,
        -0.015,
        unit_label,
        transform=axis.transAxes,
        ha="right",
        va="top",
        fontsize=9,
        color="#555555",
    )
    figure.savefig(path, dpi=300, bbox_inches="tight", facecolor="white")
    plt.close(figure)

def style_axis(axis: plt.Axes) -> None:
    """Apply restrained paper-ready axis styling."""
    axis.grid(True, color="#D9D9D9", linewidth=0.45, alpha=0.75)
    axis.set_axisbelow(True)
    axis.spines["top"].set_visible(False)
    axis.spines["right"].set_visible(False)

def render_heatmap_png(

```

```

path: Path,
times_s: np.ndarray,
radii_cm: np.ndarray,
values: np.ndarray,
title: str,
colorbar_label: str,
cmap: str,
) -> None:
    """Render a continuous time-radius field as a 300-dpi PNG heatmap."""
    configure_fonts()
    figure, axis = plt.subplots(figsize=(7.09, 3.95), constrained_layout=True)
    image = axis.pcolormesh(
        times_s / 60.0,
        radii_cm,
        values.T,
        shading="auto",
        cmap=cmap,
        rasterized=True,
    )
    colorbar = figure.colorbar(image, ax=axis, pad=0.025, aspect=28)
    colorbar.set_label(colorbar_label)
    colorbar.outline.set_linewidth(0.6)
    axis.set(
        xlabel="时间 / min",
        ylabel="到药材中心的距离 / cm",
        title=title,
        xlim=(0.0, END_TIME_S / 60.0),
        ylim=(0.0, RADIUS_M * 100.0),
    )
    axis.set_xticks(np.arange(0.0, 31.0, 5.0))
    axis.set_yticks(np.arange(0.0, 2.01, 0.5))
    figure.savefig(path, dpi=300, facecolor="white")
    plt.close(figure)

def render_radial_profiles_png(
    path: Path,
    radii_cm: np.ndarray,
    temperature_profiles: np.ndarray,
    moisture_profiles: np.ndarray,
) -> None:
    """Compare radial profiles at the seven reporting times."""
    configure_fonts()
    figure, axes = plt.subplots(
        1, 2, figsize=(7.09, 3.25), constrained_layout=True
    )
    colors = plt.get_cmap("viridis")(np.linspace(0.08, 0.92, TABLE_TIMES_S.size))

```

```

for index, (time_s, color) in enumerate(zip(TABLE_TIMES_S, colors)):
    label = f"{time_s:g} s" if time_s % 60 else f"{time_s / 60:g} min"
    axes[0].plot(
        radii_cm,
        temperature_profiles[index],
        color=color,
        linewidth=1.35,
        label=label,
    )
    axes[1].plot(
        radii_cm,
        moisture_profiles[index],
        color=color,
        linewidth=1.35,
        label=label,
    )

axes[0].set(
    title="(a) 温度径向分布",
    xlabel="到药材中心的距离 / cm",
    ylabel="温度 / °C",
    xlim=(0.0, 2.0),
)
axes[1].set(
    title="(b) 水分浓度径向分布",
    xlabel="到药材中心的距离 / cm",
    ylabel="水分浓度 / (kg/kg)",
    xlim=(0.0, 2.0),
)
for axis in axes:
    style_axis(axis)
    axis.set_xticks(np.arange(0.0, 2.01, 0.5))
axes[0].legend(frameon=False, ncol=2, loc="upper left")
axes[1].legend(frameon=False, ncol=2, loc="lower left")
figure.savefig(path, dpi=300, facecolor="white")
plt.close(figure)

def render_center_surface_evolution_png(
    path: Path,
    environment: EnvironmentData,
    times_s: np.ndarray,
    temperature_center_surface: np.ndarray,
    moisture_center_surface: np.ndarray,
) -> None:
    """Show boundary forcing and the delayed responses at centre and surface."""
    configure_fonts()

```

```

time_minutes = times_s / 60.0
oven_temperature = np.interp(
    times_s, environment.times_s, environment.temperature_c
)
oven_moisture = np.interp(
    times_s, environment.times_s, environment.moisture_kg_kg
)
figure, axes = plt.subplots(
    1, 2, figsize=(7.09, 3.25), constrained_layout=True
)

axes[0].plot(
    time_minutes,
    oven_temperature,
    color="#555555",
    linestyle="--",
    linewidth=1.2,
    label="烘房空气",
)
axes[0].plot(
    time_minutes,
    temperature_center_surface[:, 1],
    color="#D55E00",
    linewidth=1.5,
    label="药材表面",
)
axes[0].plot(
    time_minutes,
    temperature_center_surface[:, 0],
    color="#0072B2",
    linewidth=1.5,
    label="药材中心",
)
axes[0].set(
    title="(a) 中心与表面温度响应",
    xlabel="时间 / min",
    ylabel="温度 / °C",
    xlim=(0.0, 30.0),
)

axes[1].plot(
    time_minutes,
    oven_moisture,
    color="#555555",
    linestyle="--",
    linewidth=1.2,
    label="烘房空气",

```

```

)
axes[1].plot(
    time_minutes,
    moisture_center_surface[:, 1],
    color="#009E73",
    linewidth=1.5,
    label="药材表面",
)
axes[1].plot(
    time_minutes,
    moisture_center_surface[:, 0],
    color="#0072B2",
    linewidth=1.5,
    label="药材中心",
)
axes[1].set(
    title="(b) 中心与表面水分响应",
    xlabel="时间 / min",
    ylabel="水分浓度 / (kg/kg)",
    xlim=(0.0, 30.0),
)
for axis in axes:
    style_axis(axis)
    axis.set_xticks(np.arange(0.0, 31.0, 5.0))
    axis.legend(frameon=False, loc="best")
figure.savefig(path, dpi=300, facecolor="white")
plt.close(figure)

def build_validation(
    environment: EnvironmentData,
    temperature: heat.SimulationResult,
    moisture_coarse: MoistureResult,
    moisture: MoistureResult,
) -> dict:
    coarse_on_output = sample_moisture_field(
        moisture_coarse,
        moisture_coarse.times_s,
        OUTPUT_RADII_CM,
    )
    production_on_output = sample_moisture_field(
        moisture,
        moisture.times_s,
        OUTPUT_RADII_CM,
    )
    refinement_change = np.abs(coarse_on_output - production_on_output)
    coarse_table = sample_moisture_field(

```

```

    moisture_coarse,
    TABLE_TIMES_S,
    TABLE_RADII_CM,
)
production_table = sample_moisture_field(
    moisture,
    TABLE_TIMES_S,
    TABLE_RADII_CM,
)
table_refinement_change = np.abs(coarse_table - production_table)

stored_change = float(
    np.sum(
        moisture.volumes_m3_m
        * (moisture.moisture_kg_kg[-1] - moisture.moisture_kg_kg[0])
    )
)
balance_residual = stored_change + moisture.boundary_outflow_m3_kg_kg_m
balance_scale = max(abs(stored_change), abs(moisture.boundary_outflow_m3_kg_kg_m), 1e-30)

radial_violations = int(
    np.count_nonzero(np.diff(moisture.moisture_kg_kg, axis=1) > 1e-10)
    + np.count_nonzero(
        moisture.surface_moisture_kg_kg
        - moisture.moisture_kg_kg[:, -1]
        > 1e-10
    )
)
time_violations = int(
    np.count_nonzero(np.diff(moisture.moisture_kg_kg, axis=0) > 1e-10)
    + np.count_nonzero(np.diff(moisture.surface_moisture_kg_kg) > 1e-10)
)
below_environment_violations = 0
for i, time_s in enumerate(moisture.times_s):
    oven_moisture = float(
        np.interp(time_s, environment.times_s, environment.moisture_kg_kg)
    )
    below_environment_violations += int(
        np.count_nonzero(moisture.moisture_kg_kg[i] < oven_moisture - 1e-10)
    )
    below_environment_violations += int(
        moisture.surface_moisture_kg_kg[i] < oven_moisture - 1e-10
    )

all_moisture_values = np.concatenate(
    (moisture.moisture_kg_kg.ravel(), moisture.surface_moisture_kg_kg)
)

```

```

diffusivities = moisture_diffusivity(all_moisture_values)
temperature_table = sample_field(
    temperature.times_s,
    temperature.radii_m,
    temperature.temperature_c,
    TABLE_TIMES_S,
    TABLE_RADII_CM,
)
moisture_table = sample_moisture_field(
    moisture,
    TABLE_TIMES_S,
    TABLE_RADII_CM,
)

return {
    "model_scope": {
        "geometry": "one-dimensional axisymmetric cylinder",
        "temperature_equation": "transient conduction with convective boundary",
        "moisture_equation": "nonlinear Fick diffusion with convective mass boundary",
        "coupling": (
            "synchronized co-simulation; Appendix 2 supplies no latent-heat or "
            "temperature-dependent moisture term"
        ),
        "production_nominal_radial_step_cm": 0.0125,
        "production_time_step_s": 0.125,
        "coarse_nominal_radial_step_cm": 0.025,
        "coarse_time_step_s": 0.25,
        "moisture_radial_grid": "quadratically clustered toward the surface",
    },
    "moisture_grid_refinement": {
        "coarse_to_production_maximum_change_kg_kg": float(
            np.max(refinement_change)
        ),
        "coarse_to_production_mean_change_kg_kg": float(
            np.mean(refinement_change)
        ),
        "table2_maximum_change_kg_kg": float(
            np.max(table_refinement_change)
        ),
        "table2_mean_change_kg_kg": float(
            np.mean(table_refinement_change)
        ),
    },
    "moisture_balance_per_unit_length": {
        "stored_change_integral": stored_change,
        "integrated_boundary_outflow": moisture.boundary_outflow_m3_kg_kg_m,
        "residual": balance_residual,
    },
}

```

```

        "relative_residual": abs(balance_residual) / balance_scale,
    },
    "moisture_checks": {
        "minimum_kg_kg": float(np.min(all_moisture_values)),
        "maximum_kg_kg": float(np.max(all_moisture_values)),
        "center_1800_s_kg_kg": float(moisture_table[-1, 0]),
        "surface_1800_s_kg_kg": float(moisture_table[-1, -1]),
        "volume_average_1800_s_kg_kg": float(
            np.sum(moisture.volumes_m3_m * moisture.moisture_kg_kg[-1])
            / np.sum(moisture.volumes_m3_m)
        ),
        "minimum_diffusivity_m2_s": float(np.min(diffusivities)),
        "maximum_diffusivity_m2_s": float(np.max(diffusivities)),
        "radial_order_violation_count": radial_violations,
        "time_monotonicity_violation_count": time_violations,
        "below_environment_violation_count": below_environment_violations,
        "maximum_picard_iterations": moisture.maximum_picard_iterations,
    },
    "temperature_selected_outputs": {
        "center_1800_s_C": float(temperature_table[-1, 0]),
        "surface_1800_s_C": float(temperature_table[-1, -1]),
        "oven_1800_s_C": float(
            np.interp(1800.0, environment.times_s, environment.temperature_c)
        ),
    },
}
}

```

```
def main() -> None:
```

```

    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, help="Path to Attachment 1.xlsx")
    parser.add_argument("--repo-root", type=Path, help="Repository root")
    args = parser.parse_args()

    script_path = Path(__file__).resolve()
    repo_root = args.repo_root.resolve() if args.repo_root else script_path.parents[1]
    data_path = args.data.resolve() if args.data else find_default_data_path(repo_root)
    result_dir = repo_root / "results" / "A_problem1_coupled"
    picture_dir = repo_root / "picture" / "A_problem1_coupled"
    result_dir.mkdir(parents=True, exist_ok=True)
    picture_dir.mkdir(parents=True, exist_ok=True)

    environment = load_environment(data_path)

    temperature = heat.simulate_temperature(
        environment.times_s,
        environment.temperature_c,

```

```

        radial_step_cm=0.025,
        time_step_s=0.25,
    )
    moisture_coarse = simulate_moisture(
        environment.times_s,
        environment.moisture_kg_kg,
        radial_step_cm=0.025,
        time_step_s=0.25,
    )
    moisture = simulate_moisture(
        environment.times_s,
        environment.moisture_kg_kg,
        radial_step_cm=0.0125,
        time_step_s=0.125,
    )

    temperature_full = sample_field(
        temperature.times_s,
        temperature.radii_m,
        temperature.temperature_c,
        temperature.times_s,
        OUTPUT_RADII_CM,
    )
    moisture_full = sample_moisture_field(
        moisture,
        moisture.times_s,
        OUTPUT_RADII_CM,
    )
    temperature_table = sample_field(
        temperature.times_s,
        temperature.radii_m,
        temperature.temperature_c,
        TABLE_TIMES_S,
        TABLE_RADII_CM,
    )
    moisture_table = sample_moisture_field(
        moisture,
        TABLE_TIMES_S,
        TABLE_RADII_CM,
    )

    visual_radii_cm = np.linspace(0.0, 2.0, 201)
    temperature_visual = sample_field(
        temperature.times_s,
        temperature.radii_m,
        temperature.temperature_c,
        temperature.times_s,

```

```

        visual_radii_cm,
    )
    moisture_visual = sample_moisture_field(
        moisture,
        moisture.times_s,
        visual_radii_cm,
    )
    temperature_profiles = sample_field(
        temperature.times_s,
        temperature.radii_m,
        temperature.temperature_c,
        TABLE_TIMES_S,
        visual_radii_cm,
    )
    moisture_profiles = sample_moisture_field(
        moisture,
        TABLE_TIMES_S,
        visual_radii_cm,
    )
    temperature_center_surface = sample_field(
        temperature.times_s,
        temperature.radii_m,
        temperature.temperature_c,
        temperature.times_s,
        np.array([0.0, 2.0]),
    )
    moisture_center_surface = sample_moisture_field(
        moisture,
        moisture.times_s,
        np.array([0.0, 2.0]),
    )
    surface_fluxes = calculate_surface_fluxes(
        environment,
        temperature,
        moisture,
    )

    write_field_csv(
        result_dir / "temperature_full_1s_0p1cm.csv",
        temperature.times_s,
        OUTPUT_RADII_CM,
        temperature_full,
    )
    write_field_csv(
        result_dir / "moisture_full_1s_0p1cm.csv",
        moisture.times_s,
        OUTPUT_RADII_CM,

```

```

        moisture_full,
    )
    write_table_csv(
        result_dir / "table1_temperature.csv",
        TABLE_TIMES_S,
        TABLE_RADII_CM,
        temperature_table,
    )
    write_table_csv(
        result_dir / "table2_moisture.csv",
        TABLE_TIMES_S,
        TABLE_RADII_CM,
        moisture_table,
    )
    write_surface_flux_csv(
        result_dir / "surface_fluxes_1s.csv",
        surface_fluxes,
    )
    render_table_png(
        picture_dir / "table1_temperature.png",
        1,
        "30分钟内药材的温度",
        "单位: °C",
        temperature_table,
    )
    render_table_png(
        picture_dir / "table2_moisture.png",
        2,
        "30分钟内药材的水分浓度",
        "单位: kg/kg",
        moisture_table,
    )
    render_heatmap_png(
        picture_dir / "temperature_heatmap.png",
        temperature.times_s,
        visual_radii_cm,
        temperature_visual,
        "30分钟内药材温度的时空变化",
        "温度 / °C",
        "inferno",
    )
    render_heatmap_png(
        picture_dir / "moisture_heatmap.png",
        moisture.times_s,
        visual_radii_cm,
        moisture_visual,
        "30分钟内药材水分浓度的时空变化",
    )

```

```

        "水分浓度 / (kg/kg)",
        "viridis",
    )
    render_radial_profiles_png(
        picture_dir / "radial_profiles.png",
        visual_radii_cm,
        temperature_profiles,
        moisture_profiles,
    )
    render_center_surface_evolution_png(
        picture_dir / "center_surface_evolution.png",
        environment,
        temperature.times_s,
        temperature_center_surface,
        moisture_center_surface,
    )
    render_surface_flux_png(
        picture_dir / "surface_heat_moisture_flux.png",
        surface_fluxes,
    )

    validation = build_validation(
        environment, temperature, moisture_coarse, moisture
    )
    with (result_dir / "validation_summary.json").open("w", encoding="utf-8") as stream:
        json.dump(validation, stream, ensure_ascii=False, indent=2)

    print("Table 1: temperature (deg C)")
    print(np.array2string(temperature_table, precision=4, suppress_small=False))
    print("\nTable 2: moisture concentration (kg/kg)")
    print(np.array2string(moisture_table, precision=4, suppress_small=False))
    print("\nValidation summary")
    print(json.dumps(validation, ensure_ascii=False, indent=2))
    print(f"\nResults: {result_dir}")
    print(f"PNG figures: {picture_dir}")

if __name__ == "__main__":
    main()

```

附录 C 问题二代码

```

"""A problem, question 2: nonlinear coupled heat-moisture finite volumes.

```

```

The whole 0-3 h process uses the Appendix 3 constitutive laws from t=0.

```

Oven temperature and moisture are linearly interpolated from Attachment 1. Temperature is stored in degrees Celsius but converted to kelvin when the Arrhenius moisture diffusivity is evaluated. Evaporative latent heat and shrinkage are deliberately excluded in question 2.

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import csv
```

```
import json
```

```
import math
```

```
from dataclasses import dataclass
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from openpyxl import load_workbook
```

```
RADIUS_M = 0.02
```

```
END_TIME_S = 10_800.0
```

```
INITIAL_TEMPERATURE_C = 28.0
```

```
INITIAL_MOISTURE_KG_KG = 2.55
```

```
HEAT_TRANSFER_COEFFICIENT_W_M2_K = 25.0
```

```
MASS_TRANSFER_COEFFICIENT_M_S = 8.0e-7
```

```
TABLE_TIMES_S = np.array([1800, 3600, 5400, 7200, 9000, 10800], dtype=float)
```

```
TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=float)
```

```
OUTPUT_RADII_CM = np.round(np.arange(0.0, 2.0 + 0.05, 0.1), 1)
```

```
@dataclass
```

```
class EnvironmentData:
```

```
    source_times_s: np.ndarray
```

```
    source_temperature_c: np.ndarray
```

```
    source_moisture_kg_kg: np.ndarray
```

```
    times_s: np.ndarray
```

```
    temperature_c: np.ndarray
```

```
    temperature_k: np.ndarray
```

```
    moisture_kg_kg: np.ndarray
```

```
@dataclass
```

```
class Grid:
```

```
    radii_m: np.ndarray
```

```
    west_faces_m: np.ndarray
```

```
    east_faces_m: np.ndarray
```

```

volumes_m3_m: np.ndarray

@dataclass
class CoupledResult:
    times_s: np.ndarray
    grid: Grid
    temperature_c: np.ndarray
    moisture_kg_kg: np.ndarray
    surface_temperature_c: np.ndarray
    surface_moisture_kg_kg: np.ndarray
    integrated_heat_inflow_w_s_m: float
    integrated_moisture_outflow_m3_kg_kg_m: float
    maximum_picard_iterations: int

def find_default_data_path(repo_root: Path) -> Path:
    path = (
        repo_root
        / "比赛题目"
        / "CUMCM2026Problems"
        / "A题"
        / "附件"
        / "附件1.xlsx"
    )
    if not path.exists():
        raise FileNotFoundError(f"Cannot find Attachment 1: {path}")
    return path

def load_and_preprocess_environment(path: Path) -> EnvironmentData:
    """Read Attachment 1 and interpolate its 0-3 h records to every second."""
    workbook = load_workbook(path, read_only=True, data_only=True)
    sheet = workbook.active
    rows = [row[:3] for row in sheet.iter_rows(min_row=2, values_only=True) if row[0] is not
            None]
    workbook.close()

    source_times = np.asarray([float(row[0]) for row in rows], dtype=float)
    source_temperature = np.asarray([float(row[1]) for row in rows], dtype=float)
    source_moisture = np.asarray([float(row[2]) for row in rows], dtype=float)
    if not np.all(np.diff(source_times) > 0.0):
        raise ValueError("Attachment 1 time values must be strictly increasing.")
    if source_times[0] > 0.0 or source_times[-1] < END_TIME_S:
        raise ValueError("Attachment 1 does not cover the required 0-10800 s interval.")
    if not (
        np.all(np.isfinite(source_temperature))

```

```

        and np.all(np.isfinite(source_moisture))
    ):
        raise ValueError("Attachment 1 contains a non-finite environmental value.")

    times = np.arange(0.0, END_TIME_S + 1.0, 1.0)
    temperature_c = np.interp(times, source_times, source_temperature)
    moisture = np.interp(times, source_times, source_moisture)
    temperature_k = temperature_c + 273.15
    return EnvironmentData(
        source_times_s=source_times,
        source_temperature_c=source_temperature,
        source_moisture_kg_kg=source_moisture,
        times_s=times,
        temperature_c=temperature_c,
        temperature_k=temperature_k,
        moisture_kg_kg=moisture,
    )

def density_kg_m3(moisture: np.ndarray) -> np.ndarray:
    return 650.0 + 128.0 * moisture

def heat_capacity_j_kg_k(moisture: np.ndarray) -> np.ndarray:
    return 1450.0 + 2736.0 * moisture / (moisture + 1.0)

def conductivity_w_m_k(moisture: np.ndarray) -> np.ndarray:
    return 0.21 + 0.38 * moisture / (moisture + 1.0)

def moisture_diffusivity_m2_s(
    moisture: np.ndarray, temperature_c: np.ndarray
) -> np.ndarray:
    """Appendix 3:  $D=2.4e-3 \exp(-0.45/C) \exp(-3850/T)$ , T in K."""
    moisture = np.asarray(moisture, dtype=float)
    temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
    if np.any(moisture <= 0.0) or np.any(temperature_k <= 0.0):
        raise ValueError("Positive moisture and absolute temperature are required.")
    return (
        2.4e-3
        * np.exp(-0.45 / moisture)
        * np.exp(-3850.0 / temperature_k)
    )

def make_grid(nominal_radial_step_cm: float) -> Grid:

```

```

radial_step_m = nominal_radial_step_cm / 100.0
intervals = int(round(RADIUS_M / radial_step_m))
if not math.isclose(intervals * radial_step_m, RADIUS_M, abs_tol=1e-12):
    raise ValueError("The nominal radial step must divide the radius exactly.")

logical_faces = np.linspace(0.0, 1.0, intervals + 1)
faces = RADIUS_M * (1.0 - (1.0 - logical_faces) ** 2)
west_faces = faces[:-1]
east_faces = faces[1:]
radii = 0.5 * (west_faces + east_faces)
volumes = math.pi * (east_faces**2 - west_faces**2)
return Grid(radii, west_faces, east_faces, volumes)

def solve_tridiagonal(
    lower: np.ndarray,
    diagonal: np.ndarray,
    upper: np.ndarray,
    rhs: np.ndarray,
) -> np.ndarray:
    """Thomas algorithm for a strictly diagonally dominant system."""
    n = diagonal.size
    d = diagonal.copy()
    b = rhs.copy()
    u = upper.copy()
    for index in range(1, n):
        multiplier = lower[index - 1] / d[index - 1]
        d[index] -= multiplier * u[index - 1]
        b[index] -= multiplier * b[index - 1]
    if np.any(d <= 0.0):
        raise ValueError("The tridiagonal system lost positive definiteness.")
    solution = np.empty(n, dtype=float)
    solution[-1] = b[-1] / d[-1]
    for index in range(n - 2, -1, -1):
        solution[index] = (b[index] - u[index] * solution[index + 1]) / d[index]
    return solution

def internal_conductances(grid: Grid, coefficient: np.ndarray) -> np.ndarray:
    """Conductance at each internal face using two half-cell resistances."""
    face_radii = grid.east_faces_m[:-1]
    resistance = (
        (face_radii - grid.radii_m[:-1]) / coefficient[:-1]
        + (grid.radii_m[1:] - face_radii) / coefficient[1:]
    )
    return 2.0 * math.pi * face_radii / resistance

```

```

def build_diffusion_system(
    grid: Grid,
    field_previous: np.ndarray,
    diffusion_coefficient: np.ndarray,
    storage_coefficient: np.ndarray,
    boundary_transfer_coefficient: float,
    boundary_value: float,
    time_step_s: float,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, float]:
    """Assemble one backward-Euler cylindrical finite-volume system."""
    n = grid.radii_m.size
    conductances = internal_conductances(grid, diffusion_coefficient)
    storage = storage_coefficient * grid.volumes_m3_m / time_step_s
    lower = -conductances.copy()
    upper = -conductances.copy()
    diagonal = storage.copy()
    diagonal[:-1] += conductances
    diagonal[1:] += conductances
    rhs = storage * field_previous

    surface_area_per_length = 2.0 * math.pi * RADIUS_M
    boundary_conductance = surface_area_per_length / (
        (RADIUS_M - grid.radii_m[-1]) / diffusion_coefficient[-1]
        + 1.0 / boundary_transfer_coefficient
    )
    diagonal[-1] += boundary_conductance
    rhs[-1] += boundary_conductance * boundary_value
    return lower, diagonal, upper, rhs, boundary_conductance

def reconstruct_surface_value(
    cell_value: float,
    boundary_value: float,
    cell_coefficient: float,
    transfer_coefficient: float,
    cell_distance_m: float,
) -> float:
    cell_side = cell_coefficient / cell_distance_m
    return (
        cell_side * cell_value + transfer_coefficient * boundary_value
    ) / (cell_side + transfer_coefficient)

def simulate_coupled(
    environment: EnvironmentData,
    nominal_radial_step_cm: float,

```

```

time_step_s: float,
heat_transfer_coefficient_w_m2_k: float = HEAT_TRANSFER_COEFFICIENT_W_M2_K,
mass_transfer_coefficient_m_s: float = MASS_TRANSFER_COEFFICIENT_M_S,
conductivity_multiplier: float = 1.0,
diffusivity_multiplier: float = 1.0,
picard_temperature_tolerance_c: float = 1.0e-8,
picard_moisture_tolerance: float = 1.0e-10,
maximum_picard_iterations: int = 30,
) -> CoupledResult:
    """Solve the two-way property-coupled heat and moisture equations."""
    if heat_transfer_coefficient_w_m2_k <= 0.0:
        raise ValueError("The heat-transfer coefficient must be positive.")
    if mass_transfer_coefficient_m_s <= 0.0:
        raise ValueError("The mass-transfer coefficient must be positive.")
    if conductivity_multiplier <= 0.0 or diffusivity_multiplier <= 0.0:
        raise ValueError("Property multipliers must be positive.")
    steps = int(round(END_TIME_S / time_step_s))
    if not math.isclose(steps * time_step_s, END_TIME_S, abs_tol=1e-12):
        raise ValueError("The time step must divide 10800 s exactly.")
    record_stride = int(round(1.0 / time_step_s))
    if not math.isclose(record_stride * time_step_s, 1.0, abs_tol=1e-12):
        raise ValueError("The time step must divide the 1 s output interval exactly.")

    grid = make_grid(nominal_radial_step_cm)
    temperature = np.full(grid.radii_m.size, INITIAL_TEMPERATURE_C, dtype=float)
    moisture = np.full(grid.radii_m.size, INITIAL_MOISTURE_KG_KG, dtype=float)
    record_times = np.arange(0.0, END_TIME_S + 1.0, 1.0)
    temperature_records = np.empty((record_times.size, temperature.size), dtype=float)
    moisture_records = np.empty_like(temperature_records)
    surface_temperature_records = np.empty(record_times.size, dtype=float)
    surface_moisture_records = np.empty(record_times.size, dtype=float)
    temperature_records[0] = temperature
    moisture_records[0] = moisture
    surface_temperature_records[0] = INITIAL_TEMPERATURE_C
    surface_moisture_records[0] = INITIAL_MOISTURE_KG_KG

    record_index = 1
    maximum_iterations_used = 0
    integrated_heat_inflow = 0.0
    integrated_moisture_outflow = 0.0
    surface_distance = RADIUS_M - grid.radii_m[-1]

    for step in range(1, steps + 1):
        time_now = step * time_step_s
        oven_temperature = float(np.interp(time_now, environment.times_s,
            environment.temperature_c))
        oven_moisture = float(np.interp(time_now, environment.times_s,

```

```

        environment.moisture_kg_kg))
temperature_previous = temperature.copy()
moisture_previous = moisture.copy()
temperature_iterate = temperature_previous.copy()
moisture_iterate = moisture_previous.copy()

for iteration in range(1, maximum_picard_iterations + 1):
    density = density_kg_m3(moisture_iterate)
    heat_capacity = heat_capacity_j_kg_k(moisture_iterate)
    conductivity = (
        conductivity_multiplier * conductivity_w_m_k(moisture_iterate)
    )
    heat_system = build_diffusion_system(
        grid,
        temperature_previous,
        conductivity,
        density * heat_capacity,
        heat_transfer_coefficient_w_m2_k,
        oven_temperature,
        time_step_s,
    )
    temperature_updated = solve_tridiagonal(*heat_system[:4])

    diffusivity = diffusivity_multiplier * moisture_diffusivity_m2_s(
        moisture_iterate, temperature_updated
    )
    moisture_system = build_diffusion_system(
        grid,
        moisture_previous,
        diffusivity,
        np.ones_like(moisture_iterate),
        mass_transfer_coefficient_m_s,
        oven_moisture,
        time_step_s,
    )
    moisture_updated = solve_tridiagonal(*moisture_system[:4])

    temperature_error = float(
        np.max(np.abs(temperature_updated - temperature_iterate))
    )
    moisture_error = float(
        np.max(np.abs(moisture_updated - moisture_iterate))
    )
    temperature_iterate = temperature_updated
    moisture_iterate = moisture_updated
    if (
        temperature_error < picard_temperature_tolerance_c

```

```

        and moisture_error < picard_moisture_tolerance
    ):
        break
    else:
        raise RuntimeError(f"Coupled Picard iteration failed at t={time_now:g} s.")

    maximum_iterations_used = max(maximum_iterations_used, iteration)
    temperature = temperature_iterate
    moisture = moisture_iterate
    final_conductivity = conductivity_multiplier * conductivity_w_m_k(moisture)
    final_diffusivity = diffusivity_multiplier * moisture_diffusivity_m2_s(
        moisture, temperature
    )
    surface_temperature = reconstruct_surface_value(
        temperature[-1],
        oven_temperature,
        final_conductivity[-1],
        heat_transfer_coefficient_w_m2_k,
        surface_distance,
    )
    surface_moisture = reconstruct_surface_value(
        moisture[-1],
        oven_moisture,
        final_diffusivity[-1],
        mass_transfer_coefficient_m_s,
        surface_distance,
    )
    heat_boundary_conductance = build_diffusion_system(
        grid,
        temperature_previous,
        final_conductivity,
        density_kg_m3(moisture) * heat_capacity_j_kg_k(moisture),
        heat_transfer_coefficient_w_m2_k,
        oven_temperature,
        time_step_s,
    )[-1]
    moisture_boundary_conductance = build_diffusion_system(
        grid,
        moisture_previous,
        final_diffusivity,
        np.ones_like(moisture),
        mass_transfer_coefficient_m_s,
        oven_moisture,
        time_step_s,
    )[-1]
    integrated_heat_inflow += (
        time_step_s

```

```

        * heat_boundary_conductance
        * (oven_temperature - temperature[-1])
    )
    integrated_moisture_outflow += (
        time_step_s
        * moisture_boundary_conductance
        * (moisture[-1] - oven_moisture)
    )

    if step % record_stride == 0:
        temperature_records[record_index] = temperature
        moisture_records[record_index] = moisture
        surface_temperature_records[record_index] = surface_temperature
        surface_moisture_records[record_index] = surface_moisture
        record_index += 1

    return CoupledResult(
        times_s=record_times,
        grid=grid,
        temperature_c=temperature_records,
        moisture_kg_kg=moisture_records,
        surface_temperature_c=surface_temperature_records,
        surface_moisture_kg_kg=surface_moisture_records,
        integrated_heat_inflow_w_s_m=integrated_heat_inflow,
        integrated_moisture_outflow_m3_kg_kg_m=integrated_moisture_outflow,
        maximum_picard_iterations=maximum_iterations_used,
    )

def sample_result(
    result: CoupledResult,
    field_name: str,
    query_times_s: np.ndarray,
    query_radii_cm: np.ndarray,
) -> np.ndarray:
    """Sample cell-centred fields with second-order centre reconstruction."""
    if field_name == "temperature":
        field = result.temperature_c
        surface = result.surface_temperature_c
    elif field_name == "moisture":
        field = result.moisture_kg_kg
        surface = result.surface_moisture_kg_kg
    else:
        raise ValueError(f"Unknown field: {field_name}")

    time_indices = np.rint(query_times_s).astype(int)
    if not np.allclose(result.times_s[time_indices], query_times_s):

```

```

        raise ValueError("Requested times are absent from the 1 s records.")
    query_radii_m = query_radii_cm / 100.0
    extended_radii = np.concatenate(([0.0], result.grid.radii_m, [RADIUS_M]))
    sampled = np.empty((query_times_s.size, query_radii_cm.size), dtype=float)
    r0_squared = result.grid.radii_m[0] ** 2
    r1_squared = result.grid.radii_m[1] ** 2
    for row_index, time_index in enumerate(time_indices):
        values = field[time_index]
        center_value = (
            values[0] * r1_squared - values[1] * r0_squared
        ) / (r1_squared - r0_squared)
        extended_values = np.concatenate(
            ([center_value], values, [surface[time_index]])
        )
        sampled[row_index] = np.interp(
            query_radii_m, extended_radii, extended_values
        )
    return sampled

def write_environment_csv(path: Path, environment: EnvironmentData) -> None:
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(
            ["time_s", "oven_temperature_C", "oven_temperature_K", "oven_moisture_kg_kg"]
        )
        for row in zip(
            environment.times_s,
            environment.temperature_c,
            environment.temperature_k,
            environment.moisture_kg_kg,
        ):
            writer.writerow(
                [f"{row[0]:.0f}", f"{row[1]:.6f}", f"{row[2]:.6f}", f"{row[3]:.8f}"]
            )

def write_field_csv(
    path: Path, times_s: np.ndarray, radii_cm: np.ndarray, values: np.ndarray
) -> None:
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(["time_s", *[f"r_{radius:.1f}_cm" for radius in radii_cm]])
        for time_s, row in zip(times_s, values):
            writer.writerow([f"{time_s:.0f}", *[f"{value:.4f}" for value in row]])

```

```

def write_table_csv(
    path: Path,
    times_s: np.ndarray,
    radii_cm: np.ndarray,
    values: np.ndarray,
) -> None:
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(["时间/h", *[f"{radius:g} cm" for radius in radii_cm]])
        for time_s, row in zip(times_s, values):
            writer.writerow([f"{time_s / 3600:.1f}", *[f"{value:.4f}" for value in row]])

def build_validation(
    environment: EnvironmentData,
    coarse: CoupledResult,
    production: CoupledResult,
) -> dict:
    production_temperature = sample_result(
        production, "temperature", production.times_s, OUTPUT_RADII_CM
    )
    production_moisture = sample_result(
        production, "moisture", production.times_s, OUTPUT_RADII_CM
    )
    coarse_temperature = sample_result(
        coarse, "temperature", coarse.times_s, OUTPUT_RADII_CM
    )
    coarse_moisture = sample_result(
        coarse, "moisture", coarse.times_s, OUTPUT_RADII_CM
    )
    table_temperature = sample_result(
        production, "temperature", TABLE_TIMES_S, TABLE_RADII_CM
    )
    table_moisture = sample_result(
        production, "moisture", TABLE_TIMES_S, TABLE_RADII_CM
    )
    coarse_table_temperature = sample_result(
        coarse, "temperature", TABLE_TIMES_S, TABLE_RADII_CM
    )
    coarse_table_moisture = sample_result(
        coarse, "moisture", TABLE_TIMES_S, TABLE_RADII_CM
    )

    stored_moisture_change = float(
        np.sum(
            production.grid.volumes_m3_m
            * (production.moisture_kg_kg[-1] - production.moisture_kg_kg[0])
        )
    )

```

```

    )
)
moisture_balance_residual = (
    stored_moisture_change
    + production.integrated_moisture_outflow_m3_kg_kg_m
)
moisture_balance_scale = max(
    abs(stored_moisture_change),
    abs(production.integrated_moisture_outflow_m3_kg_kg_m),
    1.0e-30,
)

all_temperature = np.concatenate(
    (production.temperature_c.ravel(), production.surface_temperature_c)
)
all_moisture = np.concatenate(
    (production.moisture_kg_kg.ravel(), production.surface_moisture_kg_kg)
)
diffusivity = moisture_diffusivity_m2_s(
    production.moisture_kg_kg, production.temperature_c
)
alpha = conductivity_w_m_k(production.moisture_kg_kg) / (
    density_kg_m3(production.moisture_kg_kg)
    * heat_capacity_j_kg_k(production.moisture_kg_kg)
)
radial_temperature_increments = np.diff(production_temperature, axis=1)
# Attachment 1 contains small minute-scale temperature fluctuations near
# the plateau. These can briefly make the outermost layer a few
# thousandths of a degree cooler than its neighbour, so 0.01 C is used as
# the physically meaningful radial-order tolerance.
radial_temperature_violations = int(
    np.count_nonzero(radial_temperature_increments < -1.0e-2)
)
radial_moisture_violations = int(
    np.count_nonzero(np.diff(production.moisture_kg_kg, axis=1) > 1.0e-8)
)
time_moisture_violations = int(
    np.count_nonzero(np.diff(production.moisture_kg_kg, axis=0) > 1.0e-8)
)

return {
    "model_scope": {
        "time_interval_s": [0, 10800],
        "geometry": "fixed-radius one-dimensional axisymmetric cylinder",
        "latent_heat": "excluded as requested",
        "shrinkage": "excluded in question 2",
        "constitutive_laws": "Appendix 3 used from t=0 over the whole process",
    }
}

```

```

"environment": "Attachment 1 piecewise-linear interpolation at each step",
"production_nominal_radial_step_cm": 0.0125,
"production_time_step_s": 0.5,
"coarse_nominal_radial_step_cm": 0.025,
"coarse_time_step_s": 1.0,
},
"grid_refinement": {
    "temperature_full_max_change_C": float(
        np.max(np.abs(production_temperature - coarse_temperature))
    ),
    "temperature_full_mean_change_C": float(
        np.mean(np.abs(production_temperature - coarse_temperature))
    ),
    "moisture_full_max_change_kg_kg": float(
        np.max(np.abs(production_moisture - coarse_moisture))
    ),
    "moisture_full_mean_change_kg_kg": float(
        np.mean(np.abs(production_moisture - coarse_moisture))
    ),
    "table3_max_change_C": float(
        np.max(np.abs(table_temperature - coarse_table_temperature))
    ),
    "table4_max_change_kg_kg": float(
        np.max(np.abs(table_moisture - coarse_table_moisture))
    ),
},
"moisture_balance_per_unit_length": {
    "stored_change_integral": stored_moisture_change,
    "integrated_boundary_outflow": production.integrated_moisture_outflow_m3_kg_kg_m,
    "residual": moisture_balance_residual,
    "relative_residual": abs(moisture_balance_residual) / moisture_balance_scale,
},
"physical_ranges": {
    "temperature_min_C": float(np.min(all_temperature)),
    "temperature_max_C": float(np.max(all_temperature)),
    "moisture_min_kg_kg": float(np.min(all_moisture)),
    "moisture_max_kg_kg": float(np.max(all_moisture)),
    "diffusivity_min_m2_s": float(np.min(diffusivity)),
    "diffusivity_max_m2_s": float(np.max(diffusivity)),
    "thermal_diffusivity_min_m2_s": float(np.min(alpha)),
    "thermal_diffusivity_max_m2_s": float(np.max(alpha)),
},
"checks": {
    "radial_temperature_order_violation_count": radial_temperature_violations,
    "minimum_outward_temperature_increment_C": float(
        np.min(radial_temperature_increments)
    ),
},

```

```

        "radial_moisture_order_violation_count": radial_moisture_violations,
        "time_moisture_monotonicity_violation_count": time_moisture_violations,
        "maximum_picard_iterations": production.maximum_picard_iterations,
    },
    "selected_3h_outputs": {
        "oven_temperature_C": float(environment.temperature_c[10800]),
        "oven_moisture_kg_kg": float(environment.moisture_kg_kg[10800]),
        "center_temperature_C": float(table_temperature[-1, 0]),
        "surface_temperature_C": float(table_temperature[-1, -1]),
        "center_moisture_kg_kg": float(table_moisture[-1, 0]),
        "surface_moisture_kg_kg": float(table_moisture[-1, -1]),
        "volume_average_moisture_kg_kg": float(
            np.sum(
                production.grid.volumes_m3_m
                * production.moisture_kg_kg[-1]
            )
            / np.sum(production.grid.volumes_m3_m)
        ),
    },
}
}

```

```

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, help="Path to Attachment 1.xlsx")
    parser.add_argument("--repo-root", type=Path, help="Repository root")
    args = parser.parse_args()

    script_path = Path(__file__).resolve()
    repo_root = args.repo_root.resolve() if args.repo_root else script_path.parents[1]
    data_path = args.data.resolve() if args.data else find_default_data_path(repo_root)
    result_dir = repo_root / "results" / "A_problem2_coupled"
    result_dir.mkdir(parents=True, exist_ok=True)

    environment = load_and_preprocess_environment(data_path)
    write_environment_csv(result_dir / "environment_0_3h_1s_kelvin.csv", environment)

    coarse = simulate_coupled(
        environment,
        nominal_radial_step_cm=0.025,
        time_step_s=1.0,
    )
    production = simulate_coupled(
        environment,
        nominal_radial_step_cm=0.0125,
        time_step_s=0.5,
    )

```

```

temperature_full = sample_result(
    production, "temperature", production.times_s, OUTPUT_RADII_CM
)
moisture_full = sample_result(
    production, "moisture", production.times_s, OUTPUT_RADII_CM
)
temperature_table = sample_result(
    production, "temperature", TABLE_TIMES_S, TABLE_RADII_CM
)
moisture_table = sample_result(
    production, "moisture", TABLE_TIMES_S, TABLE_RADII_CM
)

write_field_csv(
    result_dir / "temperature_full_1s_0p1cm.csv",
    production.times_s,
    OUTPUT_RADII_CM,
    temperature_full,
)
write_field_csv(
    result_dir / "moisture_full_1s_0p1cm.csv",
    production.times_s,
    OUTPUT_RADII_CM,
    moisture_full,
)
write_table_csv(
    result_dir / "table3_temperature.csv",
    TABLE_TIMES_S,
    TABLE_RADII_CM,
    temperature_table,
)
write_table_csv(
    result_dir / "table4_moisture.csv",
    TABLE_TIMES_S,
    TABLE_RADII_CM,
    moisture_table,
)
validation = build_validation(environment, coarse, production)
with (result_dir / "validation_summary.json").open("w", encoding="utf-8") as stream:
    json.dump(validation, stream, ensure_ascii=False, indent=2)

print("Table 3: temperature (deg C)")
print(np.array2string(temperature_table, precision=4, suppress_small=False))
print("\nTable 4: moisture concentration (kg/kg)")
print(np.array2string(moisture_table, precision=4, suppress_small=False))
print("\nValidation summary")

```

```

print(json.dumps(validation, ensure_ascii=False, indent=2))
print(f"\nResults: {result_dir}")

if __name__ == "__main__":
    main()

```

附录 D 问题三代码

```

"""A problem, question 3: full-process drying time by coupled finite volumes.

The model continues the Appendix 3 heat-moisture equations until every radial
position has a dry-basis moisture concentration below 0.15 kg/kg. Attachment
1 drives the boundary from 0 to 4 h. Thereafter, the oven temperature and
moisture are held at their time-weighted averages over the final measured hour
(3-4 h). Shrinkage and evaporation latent heat are excluded in question 3.
"""

from __future__ import annotations

import argparse
import csv
import json
import math
from dataclasses import dataclass
from pathlib import Path

import numpy as np

import a2_coupled_fvm as q2

MEASURED_END_TIME_S = 4.0 * 3600.0
PLATEAU_START_TIME_S = 3.0 * 3600.0
DRYING_THRESHOLD_KG_KG = 0.15
OUTPUT_INTERVAL_S = 60.0
TABLE_INTERVAL_S = 6.0 * 3600.0
MAXIMUM_SIMULATION_TIME_S = 7.0 * 24.0 * 3600.0
OUTPUT_RADII_CM = np.round(np.arange(0.0, 2.0 + 0.05, 0.1), 1)
TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=float)

@dataclass
class BoundaryProgram:
    source_times_s: np.ndarray

```

```

source_temperature_c: np.ndarray
source_moisture_kg_kg: np.ndarray
plateau_temperature_c: float
plateau_moisture_kg_kg: float

def values(self, time_s: float) -> tuple[float, float]:
    if time_s <= MEASURED_END_TIME_S:
        return (
            float(
                np.interp(
                    time_s, self.source_times_s, self.source_temperature_c
                )
            ),
            float(
                np.interp(
                    time_s, self.source_times_s, self.source_moisture_kg_kg
                )
            ),
        )
    return self.plateau_temperature_c, self.plateau_moisture_kg_kg

```

@dataclass

```

class LongDryingResult:
    times_s: np.ndarray
    temperature_c: np.ndarray
    moisture_kg_kg: np.ndarray
    drying_time_s: float
    last_not_dry_time_s: float
    last_not_dry_maximum_moisture: float
    final_maximum_moisture: float
    controlling_radius_cm: float
    final_internal_temperature_c: np.ndarray
    final_internal_moisture_kg_kg: np.ndarray
    grid: q2.Grid
    integrated_moisture_outflow: float
    maximum_picard_iterations: int
    time_step_counts: dict[str, int]

```

@dataclass

```

class StepResult:
    temperature_c: np.ndarray
    moisture_kg_kg: np.ndarray
    surface_temperature_c: float
    surface_moisture_kg_kg: float
    heat_boundary_conductance: float

```

```

moisture_boundary_conductance: float
iterations: int

def build_boundary_program(environment: q2.EnvironmentData) -> BoundaryProgram:
    """Use a time-weighted 3-4 h mean as the post-measurement plateau."""
    mask = (
        (environment.source_times_s >= PLATEAU_START_TIME_S)
        & (environment.source_times_s <= MEASURED_END_TIME_S)
    )
    times = environment.source_times_s[mask]
    temperature = environment.source_temperature_c[mask]
    moisture = environment.source_moisture_kg_kg[mask]
    if times.size < 2 or times[0] != PLATEAU_START_TIME_S or times[-1] != MEASURED_END_TIME_S:
        raise ValueError("Attachment 1 must contain the complete 3-4 h plateau interval.")
    duration = MEASURED_END_TIME_S - PLATEAU_START_TIME_S
    plateau_temperature = float(np.trapezoid(temperature, times) / duration)
    plateau_moisture = float(np.trapezoid(moisture, times) / duration)
    return BoundaryProgram(
        source_times_s=environment.source_times_s,
        source_temperature_c=environment.source_temperature_c,
        source_moisture_kg_kg=environment.source_moisture_kg_kg,
        plateau_temperature_c=plateau_temperature,
        plateau_moisture_kg_kg=plateau_moisture,
    )

def boundary_conductance(
    grid: q2.Grid, cell_coefficient: float, transfer_coefficient: float
) -> float:
    surface_area_per_length = 2.0 * math.pi * q2.RADIUS_M
    return surface_area_per_length / (
        (q2.RADIUS_M - grid.radii_m[-1]) / cell_coefficient
        + 1.0 / transfer_coefficient
    )

def advance_coupled_step(
    grid: q2.Grid,
    temperature_previous: np.ndarray,
    moisture_previous: np.ndarray,
    oven_temperature_c: float,
    oven_moisture_kg_kg: float,
    time_step_s: float,
    temperature_tolerance_c: float = 1.0e-8,
    moisture_tolerance: float = 1.0e-10,
    maximum_iterations: int = 30,

```

```

) -> StepResult:
    """Advance one fully coupled implicit time step by Picard iteration."""
    temperature_iterate = temperature_previous.copy()
    moisture_iterate = moisture_previous.copy()
    for iteration in range(1, maximum_iterations + 1):
        conductivity = q2.conductivity_w_m_k(moisture_iterate)
        volumetric_heat_capacity = (
            q2.density_kg_m3(moisture_iterate)
            * q2.heat_capacity_j_kg_k(moisture_iterate)
        )
        heat_system = q2.build_diffusion_system(
            grid,
            temperature_previous,
            conductivity,
            volumetric_heat_capacity,
            q2.HEAT_TRANSFER_COEFFICIENT_W_M2_K,
            oven_temperature_c,
            time_step_s,
        )
        temperature_updated = q2.solve_tridiagonal(*heat_system[:4])

        diffusivity = q2.moisture_diffusivity_m2_s(
            moisture_iterate, temperature_updated
        )
        moisture_system = q2.build_diffusion_system(
            grid,
            moisture_previous,
            diffusivity,
            np.ones_like(moisture_iterate),
            q2.MASS_TRANSFER_COEFFICIENT_M_S,
            oven_moisture_kg_kg,
            time_step_s,
        )
        moisture_updated = q2.solve_tridiagonal(*moisture_system[:4])

        temperature_error = float(
            np.max(np.abs(temperature_updated - temperature_iterate))
        )
        moisture_error = float(np.max(np.abs(moisture_updated - moisture_iterate)))
        temperature_iterate = temperature_updated
        moisture_iterate = moisture_updated
        if (
            temperature_error < temperature_tolerance_c
            and moisture_error < moisture_tolerance
        ):
            break
    else:

```

```

        raise RuntimeError("Coupled Picard iteration did not converge.")

    final_conductivity = q2.conductivity_w_m_k(moisture_iterate)
    final_diffusivity = q2.moisture_diffusivity_m2_s(
        moisture_iterate, temperature_iterate
    )
    surface_distance = q2.RADIUS_M - grid.radii_m[-1]
    surface_temperature = q2.reconstruct_surface_value(
        temperature_iterate[-1],
        oven_temperature_c,
        final_conductivity[-1],
        q2.HEAT_TRANSFER_COEFFICIENT_W_M2_K,
        surface_distance,
    )
    surface_moisture = q2.reconstruct_surface_value(
        moisture_iterate[-1],
        oven_moisture_kg_kg,
        final_diffusivity[-1],
        q2.MASS_TRANSFER_COEFFICIENT_M_S,
        surface_distance,
    )
    return StepResult(
        temperature_c=temperature_iterate,
        moisture_kg_kg=moisture_iterate,
        surface_temperature_c=surface_temperature,
        surface_moisture_kg_kg=surface_moisture,
        heat_boundary_conductance=boundary_conductance(
            grid,
            float(final_conductivity[-1]),
            q2.HEAT_TRANSFER_COEFFICIENT_W_M2_K,
        ),
        moisture_boundary_conductance=boundary_conductance(
            grid,
            float(final_diffusivity[-1]),
            q2.MASS_TRANSFER_COEFFICIENT_M_S,
        ),
        iterations=iteration,
    )

def reconstruct_center(grid: q2.Grid, values: np.ndarray) -> float:
    """Second-order even-symmetry reconstruction at r=0."""
    r0_squared = grid.radii_m[0] ** 2
    r1_squared = grid.radii_m[1] ** 2
    return float(
        (values[0] * r1_squared - values[1] * r0_squared)
        / (r1_squared - r0_squared)
    )

```

```

)

def sample_radial_field(
    grid: q2.Grid,
    values: np.ndarray,
    surface_value: float,
    query_radii_cm: np.ndarray,
) -> np.ndarray:
    extended_radii = np.concatenate(([0.0], grid.radii_m, [q2.RADIUS_M]))
    extended_values = np.concatenate(
        ([reconstruct_center(grid, values)], values, [surface_value])
    )
    return np.interp(query_radii_cm / 100.0, extended_radii, extended_values)

def choose_time_step(
    time_s: float,
    maximum_moisture: float,
    early_time_step_s: float,
    late_time_step_s: float,
    final_time_step_s: float,
    refinement_margin: float = 0.002,
) -> tuple[float, str]:
    if time_s < MEASURED_END_TIME_S:
        return early_time_step_s, "measured_environment"
    if maximum_moisture < DRYING_THRESHOLD_KG_KG + refinement_margin:
        return final_time_step_s, "threshold_refinement"
    return late_time_step_s, "constant_environment"

def simulate_until_dry(
    boundary: BoundaryProgram,
    nominal_radial_step_cm: float,
    early_time_step_s: float,
    late_time_step_s: float,
    final_time_step_s: float,
    refinement_margin: float = 0.002,
) -> LongDryingResult:
    """Run the full coupled model until all radial positions are dry."""
    grid = q2.make_grid(nominal_radial_step_cm)
    temperature = np.full(
        grid.radii_m.size, q2.INITIAL_TEMPERATURE_C, dtype=float
    )
    moisture = np.full(
        grid.radii_m.size, q2.INITIAL_MOISTURE_KG_KG, dtype=float
    )

```

```

surface_temperature = q2.INITIAL_TEMPERATURE_C
surface_moisture = q2.INITIAL_MOISTURE_KG_KG
time_s = 0.0
next_output_time_s = OUTPUT_INTERVAL_S
record_times: list[float] = []
temperature_records: list[np.ndarray] = []
moisture_records: list[np.ndarray] = []
integrated_moisture_outflow = 0.0
maximum_picard_iterations = 0
time_step_counts = {
    "measured_environment": 0,
    "constant_environment": 0,
    "threshold_refinement": 0,
}

maximum_moisture = q2.INITIAL_MOISTURE_KG_KG
last_not_dry_time = 0.0
last_not_dry_maximum = maximum_moisture
controlling_radius_cm = 0.0

while time_s < MAXIMUM_SIMULATION_TIME_S:
    proposed_step, step_label = choose_time_step(
        time_s,
        maximum_moisture,
        early_time_step_s,
        late_time_step_s,
        final_time_step_s,
        refinement_margin,
    )
    time_step_s = proposed_step
    if time_s < MEASURED_END_TIME_S < time_s + time_step_s:
        time_step_s = MEASURED_END_TIME_S - time_s
    if time_s < next_output_time_s < time_s + time_step_s:
        time_step_s = next_output_time_s - time_s
    if time_s + time_step_s > MAXIMUM_SIMULATION_TIME_S:
        time_step_s = MAXIMUM_SIMULATION_TIME_S - time_s
    if time_step_s <= 0.0:
        raise RuntimeError("Non-positive adaptive time step encountered.")

    new_time = time_s + time_step_s
    oven_temperature, oven_moisture = boundary.values(new_time)
    step = advance_coupled_step(
        grid,
        temperature,
        moisture,
        oven_temperature,
        oven_moisture,

```

```

        time_step_s,
    )
    integrated_moisture_outflow += (
        time_step_s
        * step.moisture_boundary_conductance
        * (step.moisture_kg_kg[-1] - oven_moisture)
    )
    temperature = step.temperature_c
    moisture = step.moisture_kg_kg
    surface_temperature = step.surface_temperature_c
    surface_moisture = step.surface_moisture_kg_kg
    time_s = new_time
    time_step_counts[step_label] += 1
    maximum_picard_iterations = max(
        maximum_picard_iterations, step.iterations
    )

    center_moisture = reconstruct_center(grid, moisture)
    extended_moisture = np.concatenate(
        ([center_moisture], moisture, [surface_moisture])
    )
    maximum_index = int(np.argmax(extended_moisture))
    maximum_moisture = float(extended_moisture[maximum_index])
    extended_radii_cm = np.concatenate(
        ([0.0], grid.radii_m * 100.0, [q2.RADIUS_M * 100.0])
    )
    controlling_radius_cm = float(extended_radii_cm[maximum_index])

    is_output_time = math.isclose(
        time_s, next_output_time_s, abs_tol=1.0e-8
    )
    if is_output_time:
        record_times.append(time_s)
        temperature_records.append(
            sample_radial_field(
                grid,
                temperature,
                surface_temperature,
                OUTPUT_RADII_CM,
            )
        )
        moisture_records.append(
            sample_radial_field(
                grid,
                moisture,
                surface_moisture,
                OUTPUT_RADII_CM,

```

```

        )
    )
    next_output_time_s += OUTPUT_INTERVAL_S

    if maximum_moisture < DRYING_THRESHOLD_KG_KG:
        if not is_output_time:
            record_times.append(time_s)
            temperature_records.append(
                sample_radial_field(
                    grid,
                    temperature,
                    surface_temperature,
                    OUTPUT_RADII_CM,
                )
            )
            moisture_records.append(
                sample_radial_field(
                    grid,
                    moisture,
                    surface_moisture,
                    OUTPUT_RADII_CM,
                )
            )
        )
        break

    last_not_dry_time = time_s
    last_not_dry_maximum = maximum_moisture
else:
    raise RuntimeError("Maximum simulation time reached before drying.")

return LongDryingResult(
    times_s=np.asarray(record_times, dtype=float),
    temperature_c=np.vstack(temperature_records),
    moisture_kg_kg=np.vstack(moisture_records),
    drying_time_s=time_s,
    last_not_dry_time_s=last_not_dry_time,
    last_not_dry_maximum_moisture=last_not_dry_maximum,
    final_maximum_moisture=maximum_moisture,
    controlling_radius_cm=controlling_radius_cm,
    final_internal_temperature_c=temperature,
    final_internal_moisture_kg_kg=moisture,
    grid=grid,
    integrated_moisture_outflow=integrated_moisture_outflow,
    maximum_picard_iterations=maximum_picard_iterations,
    time_step_counts=time_step_counts,
)

```

```

def values_at_times(result: LongDryingResult, query_times_s: np.ndarray) -> np.ndarray:
    """Return recorded 0.1 cm moisture rows at exact requested times."""
    result_lookup = {round(float(time), 8): index for index, time in enumerate(result.times_s)}
    rows = []
    for time in query_times_s:
        key = round(float(time), 8)
        if key not in result_lookup:
            raise ValueError(f"Requested time {time:g} s was not recorded.")
        rows.append(result.moisture_kg_kg[result_lookup[key]])
    return np.vstack(rows)

def build_table_times(drying_time_s: float) -> np.ndarray:
    regular = np.arange(
        TABLE_INTERVAL_S,
        drying_time_s - 1.0e-9,
        TABLE_INTERVAL_S,
        dtype=float,
    )
    if regular.size and math.isclose(regular[-1], drying_time_s, abs_tol=1.0e-8):
        return regular
    return np.append(regular, drying_time_s)

def table_values(result: LongDryingResult, table_times_s: np.ndarray) -> np.ndarray:
    output_values = values_at_times(result, table_times_s)
    column_indices = [
        int(np.where(np.isclose(OUTPUT_RADII_CM, radius))[0][0])
        for radius in TABLE_RADII_CM
    ]
    return output_values[:, column_indices]

def write_full_csv(path: Path, result: LongDryingResult) -> None:
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(["time_s", *[f"r_{r:.1f}_cm" for r in OUTPUT_RADII_CM]])
        for time_s, row in zip(result.times_s, result.moisture_kg_kg):
            writer.writerow([f"{time_s:.0f}", *[f"{value:.4f}" for value in row]])

def write_temperature_csv(path: Path, result: LongDryingResult) -> None:
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(["time_s", *[f"r_{r:.1f}_cm" for r in OUTPUT_RADII_CM]])
        for time_s, row in zip(result.times_s, result.temperature_c):

```

```

        writer.writerow([f"{time_s:.0f}", *[f"{value:.4f}" for value in row]])

def write_table_csv(
    path: Path, table_times_s: np.ndarray, values: np.ndarray, drying_time_s: float
) -> None:
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(["时间/h", *[f"{radius:g} cm" for radius in TABLE_RADII_CM]])
        for time_s, row in zip(table_times_s, values):
            label = (
                "烘干结束时间"
                if math.isclose(time_s, drying_time_s, abs_tol=1.0e-8)
                else f"{time_s / 3600:.0f}"
            )
            writer.writerow([label, *[f"{value:.4f}" for value in row]])

def build_validation(
    boundary: BoundaryProgram,
    production: LongDryingResult,
    coarse: LongDryingResult,
    table_times_s: np.ndarray,
    production_table: np.ndarray,
) -> dict:
    common_end = min(production.drying_time_s, coarse.drying_time_s)
    common_times = production.times_s[
        (production.times_s <= common_end)
        & np.isclose(np.mod(production.times_s, OUTPUT_INTERVAL_S), 0.0)
    ]
    coarse_lookup = {round(float(t), 8): i for i, t in enumerate(coarse.times_s)}
    common_times = np.asarray(
        [t for t in common_times if round(float(t), 8) in coarse_lookup],
        dtype=float,
    )
    production_common = values_at_times(production, common_times)
    coarse_common = values_at_times(coarse, common_times)
    refinement_change = np.abs(production_common - coarse_common)

    stored_change = float(
        np.sum(
            production.grid.volumes_m3_m
            * (
                production.final_internal_moisture_kg_kg
                - q2.INITIAL_MOISTURE_KG_KG
            )
        )
    )

```

```

)
balance_residual = stored_change + production.integrated_moisture_outflow
balance_scale = max(
    abs(stored_change), abs(production.integrated_moisture_outflow), 1.0e-30
)
radial_violations = int(
    np.count_nonzero(np.diff(production.moisture_kg_kg, axis=1) > 1.0e-8)
)
time_violations = int(
    np.count_nonzero(np.diff(production.moisture_kg_kg, axis=0) > 1.0e-8)
)
final_row = production.moisture_kg_kg[-1]
final_max_output = float(np.max(final_row))

return {
    "model_scope": {
        "geometry": "fixed-radius one-dimensional axisymmetric cylinder",
        "constitutive_laws": "Appendix 3 used over the full process",
        "latent_heat": "excluded",
        "shrinkage": "excluded in question 3",
        "drying_threshold_kg_kg": DRYING_THRESHOLD_KG_KG,
        "measured_boundary_interval_s": [0, int(MEASURED_END_TIME_S)],
        "post_4h_boundary": "time-weighted mean over Attachment 1 final hour",
        "production_nominal_radial_step_cm": 0.0125,
        "production_time_steps_s": [1.0, 30.0, 1.0],
        "coarse_nominal_radial_step_cm": 0.025,
        "coarse_time_steps_s": [2.0, 60.0, 2.0],
        "saved_interval_s": OUTPUT_INTERVAL_S,
    },
    "constant_boundary": {
        "temperature_C": boundary.plateau_temperature_c,
        "temperature_K": boundary.plateau_temperature_c + 273.15,
        "moisture_kg_kg": boundary.plateau_moisture_kg_kg,
    },
    "drying_time": {
        "production_s": production.drying_time_s,
        "production_h": production.drying_time_s / 3600.0,
        "coarse_s": coarse.drying_time_s,
        "coarse_h": coarse.drying_time_s / 3600.0,
        "coarse_production_difference_s": abs(
            production.drying_time_s - coarse.drying_time_s
        ),
        "controlling_radius_cm": production.controlling_radius_cm,
        "previous_time_s": production.last_not_dry_time_s,
        "previous_maximum_moisture": production.last_not_dry_maximum_moisture,
        "final_maximum_moisture": production.final_maximum_moisture,
        "final_output_grid_maximum_moisture": final_max_output,
    },
}

```

```

    },
    "grid_refinement": {
        "common_record_count": int(common_times.size),
        "maximum_moisture_change_kg_kg": float(np.max(refinement_change)),
        "mean_moisture_change_kg_kg": float(np.mean(refinement_change)),
    },
    "moisture_balance_per_unit_length": {
        "stored_change_integral": stored_change,
        "integrated_boundary_outflow": production.integrated_moisture_outflow,
        "residual": balance_residual,
        "relative_residual": abs(balance_residual) / balance_scale,
    },
    "checks": {
        "previous_state_not_dry": bool(
            production.last_not_dry_maximum_moisture
            >= DRYING_THRESHOLD_KG_KG
        ),
        "final_state_all_dry": bool(
            production.final_maximum_moisture < DRYING_THRESHOLD_KG_KG
        ),
        "radial_order_violation_count": radial_violations,
        "time_monotonicity_violation_count": time_violations,
        "minimum_saved_moisture_kg_kg": float(
            np.min(production.moisture_kg_kg)
        ),
        "maximum_picard_iterations": production.maximum_picard_iterations,
        "time_step_counts": production.time_step_counts,
    },
    "table5": {
        "row_count": int(table_times_s.size),
        "last_time_h": float(table_times_s[-1] / 3600.0),
        "last_row": [float(value) for value in production_table[-1]],
    },
}
}

```

```
def main() -> None:
```

```

    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, help="Path to Attachment 1.xlsx")
    parser.add_argument("--repo-root", type=Path, help="Repository root")
    args = parser.parse_args()

    script_path = Path(__file__).resolve()
    repo_root = args.repo_root.resolve() if args.repo_root else script_path.parents[1]
    data_path = args.data.resolve() if args.data else q2.find_default_data_path(repo_root)
    result_dir = repo_root / "results" / "A_problem3_drying_time"
    result_dir.mkdir(parents=True, exist_ok=True)

```

```

environment = q2.load_and_preprocess_environment(data_path)
boundary = build_boundary_program(environment)
coarse = simulate_until_dry(
    boundary,
    nominal_radial_step_cm=0.025,
    early_time_step_s=2.0,
    late_time_step_s=60.0,
    final_time_step_s=2.0,
)
production = simulate_until_dry(
    boundary,
    nominal_radial_step_cm=0.0125,
    early_time_step_s=1.0,
    late_time_step_s=30.0,
    final_time_step_s=1.0,
)

table_times = build_table_times(production.drying_time_s)
table = table_values(production, table_times)
write_full_csv(result_dir / "moisture_full_60s_0p1cm.csv", production)
write_temperature_csv(result_dir / "temperature_auxiliary_60s_0p1cm.csv", production)
write_table_csv(
    result_dir / "table5_moisture.csv",
    table_times,
    table,
    production.drying_time_s,
)
validation = build_validation(
    boundary, production, coarse, table_times, table
)
with (result_dir / "validation_summary.json").open("w", encoding="utf-8") as stream:
    json.dump(validation, stream, ensure_ascii=False, indent=2)

print(
    f"Plateau boundary: {boundary.plateau_temperature_c:.6f} deg C, "
    f"{boundary.plateau_moisture_kg_kg:.8f} kg/kg"
)
print(
    f"Drying time: {production.drying_time_s:.0f} s "
    f"= {production.drying_time_s / 3600.0:.6f} h"
)
print(
    f"Threshold bracket: {production.last_not_dry_time_s:.0f} s -> "
    f"{production.drying_time_s:.0f} s; "
    f"{production.last_not_dry_maximum_moisture:.10f} -> "
    f"{production.final_maximum_moisture:.10f} kg/kg"
)

```

```

)
print("\nTable 5: moisture concentration (kg/kg)")
for time_s, row in zip(table_times, table):
    label = "end" if math.isclose(time_s, production.drying_time_s) else f"{time_s /
        3600:.0f} h"
    print(label, np.array2string(row, precision=4, suppress_small=False))
print("\nValidation summary")
print(json.dumps(validation, ensure_ascii=False, indent=2))
print(f"\nResults: {result_dir}")

if __name__ == "__main__":
    main()

```

附录 E 问题四代码

```

"""A problem, question 4: moving-radius heat-moisture finite-volume model.

The cylinder radius is read from Attachment 2 and linearly interpolated in
time. Uniform radial shrinkage is assumed, so  $\xi=r/R(t)$  labels a material
position. The material derivative therefore becomes a time derivative at
fixed  $\xi$ , while the current radius remains in the physical finite-volume
geometry. Appendix 4 properties are used throughout. Temperature is stored
and advanced in kelvin, including the Arrhenius diffusivity evaluation.

The requested result4 grid is reported at fixed physical distances
0.0, 0.1, ..., 2.0 cm plus a separate moving-surface column. A fixed-distance
cell is left blank after the shrinking surface has moved inside that distance.
"""

from __future__ import annotations

import argparse
import csv
import json
import math
from dataclasses import dataclass
from pathlib import Path

import numpy as np
from openpyxl import load_workbook

import a2_coupled_fvm as q2

```

```

INITIAL_RADIUS_M = 0.02
INITIAL_TEMPERATURE_K = 28.0 + 273.15
INITIAL_MOISTURE_KG_KG = 2.55
HEAT_TRANSFER_COEFFICIENT_W_M2_K = 25.0
MASS_TRANSFER_COEFFICIENT_M_S = 8.0e-7

MEASURED_ENVIRONMENT_END_S = 4.0 * 3600.0
PLATEAU_START_S = 3.0 * 3600.0
DRYING_THRESHOLD_KG_KG = 0.15
OUTPUT_INTERVAL_S = 60.0
TABLE_INTERVAL_S = 6.0 * 3600.0
MAXIMUM_SIMULATION_TIME_S = 7.0 * 24.0 * 3600.0

OUTPUT_RADII_CM = np.round(np.arange(0.0, 2.0 + 0.05, 0.1), 1)
TABLE_RADII_CM = np.round(np.arange(0.0, 2.0 + 0.25, 0.5), 1)

@dataclass(frozen=True)
class BoundaryProgram:
    source_times_s: np.ndarray
    source_temperature_k: np.ndarray
    source_moisture_kg_kg: np.ndarray
    plateau_temperature_k: float
    plateau_moisture_kg_kg: float

    def values(self, time_s: float) -> tuple[float, float]:
        if time_s <= MEASURED_ENVIRONMENT_END_S:
            return (
                float(
                    np.interp(time_s, self.source_times_s, self.source_temperature_k)
                ),
                float(
                    np.interp(
                        time_s,
                        self.source_times_s,
                        self.source_moisture_kg_kg,
                    )
                ),
            )
        return self.plateau_temperature_k, self.plateau_moisture_kg_kg

@dataclass(frozen=True)
class RadiusProgram:
    source_times_s: np.ndarray
    source_radii_cm: np.ndarray

```

```

def radius_cm(self, time_s: float) -> float:
    return float(
        np.interp(
            time_s,
            self.source_times_s,
            self.source_radii_cm,
            left=self.source_radii_cm[0],
            right=self.source_radii_cm[-1],
        )
    )

def radius_m(self, time_s: float) -> float:
    return self.radius_cm(time_s) / 100.0

@dataclass(frozen=True)
class MaterialGrid:
    centers_xi: np.ndarray
    west_faces_xi: np.ndarray
    east_faces_xi: np.ndarray

@dataclass
class StepResult:
    temperature_k: np.ndarray
    moisture_kg_kg: np.ndarray
    surface_temperature_k: float
    surface_moisture_kg_kg: float
    moisture_boundary_conductance: float
    moisture_storage_change: float
    minimum_diffusivity_m2_s: float
    maximum_diffusivity_m2_s: float
    iterations: int

@dataclass
class ShrinkageResult:
    times_s: np.ndarray
    radii_cm: np.ndarray
    temperature_k: np.ndarray
    moisture_kg_kg: np.ndarray
    surface_temperature_k: np.ndarray
    surface_moisture_kg_kg: np.ndarray
    drying_time_s: float
    previous_time_s: float
    previous_maximum_moisture: float
    final_maximum_moisture: float

```

```

controlling_radius_cm: float
final_internal_temperature_k: np.ndarray
final_internal_moisture_kg_kg: np.ndarray
maximum_picard_iterations: int
time_step_counts: dict[str, int]
discrete_moisture_storage_change: float
integrated_moisture_outflow: float
minimum_diffusivity_m2_s: float
maximum_diffusivity_m2_s: float

def find_attachment(repo_root: Path, filename: str) -> Path:
    path = (
        repo_root
        / "比赛题目"
        / "CUMCM2026Problems"
        / "A题"
        / "附件"
        / filename
    )
    if not path.exists():
        raise FileNotFoundError(f"Cannot find {filename}: {path}")
    return path

def load_radius_program(path: Path) -> RadiusProgram:
    workbook = load_workbook(path, read_only=True, data_only=True)
    sheet = workbook.active
    rows = [
        row[:2]
        for row in sheet.iter_rows(min_row=2, values_only=True)
        if row[0] is not None
    ]
    workbook.close()
    times = np.asarray([float(row[0]) for row in rows], dtype=float)
    radii = np.asarray([float(row[1]) for row in rows], dtype=float)
    if times.size < 2 or not np.all(np.diff(times) > 0.0):
        raise ValueError("Attachment 2 time values must be strictly increasing.")
    if not math.isclose(times[0], 0.0, abs_tol=1.0e-12):
        raise ValueError("Attachment 2 must start at t=0 s.")
    if not math.isclose(radii[0] / 100.0, INITIAL_RADIUS_M, abs_tol=1.0e-12):
        raise ValueError("Attachment 2 initial radius must be 2 cm.")
    if np.any(radii <= 0.0) or np.any(np.diff(radii) > 1.0e-12):
        raise ValueError("Attachment 2 radii must remain positive and non-increasing.")
    if not np.all(np.isfinite(radii)):
        raise ValueError("Attachment 2 contains a non-finite radius.")
    return RadiusProgram(times, radii)

```

```

def build_boundary_program(environment: q2.EnvironmentData) -> BoundaryProgram:
    """Use Attachment 1 through 4 h and its time-weighted 3-4 h mean later."""
    mask = (
        (environment.source_times_s >= PLATEAU_START_S)
        & (environment.source_times_s <= MEASURED_ENVIRONMENT_END_S)
    )
    times = environment.source_times_s[mask]
    temperature_k = environment.source_temperature_c[mask] + 273.15
    moisture = environment.source_moisture_kg_kg[mask]
    if (
        times.size < 2
        or not math.isclose(times[0], PLATEAU_START_S)
        or not math.isclose(times[-1], MEASURED_ENVIRONMENT_END_S)
    ):
        raise ValueError("Attachment 1 must contain the complete 3-4 h interval.")
    duration = MEASURED_ENVIRONMENT_END_S - PLATEAU_START_S
    return BoundaryProgram(
        source_times_s=environment.source_times_s,
        source_temperature_k=environment.source_temperature_c + 273.15,
        source_moisture_kg_kg=environment.source_moisture_kg_kg,
        plateau_temperature_k=float(np.trapezoid(temperature_k, times) / duration),
        plateau_moisture_kg_kg=float(np.trapezoid(moisture, times) / duration),
    )

def density_kg_m3(moisture: np.ndarray) -> np.ndarray:
    """Appendix 4 bulk density."""
    return 760.0 + 90.0 * np.asarray(moisture, dtype=float)

def heat_capacity_j_kg_k(moisture: np.ndarray) -> np.ndarray:
    """Appendix 4 specific heat capacity."""
    moisture = np.asarray(moisture, dtype=float)
    return 1850.0 + 2150.0 * moisture / (moisture + 1.0)

def conductivity_w_m_k(moisture: np.ndarray) -> np.ndarray:
    """Appendix 4 thermal conductivity."""
    moisture = np.asarray(moisture, dtype=float)
    return 0.12 + 0.20 * moisture / (moisture + 1.0)

def moisture_diffusivity_m2_s(
    moisture: np.ndarray, temperature_k: np.ndarray
) -> np.ndarray:

```

```

"""Appendix 4:  $D=4.2e-4 \exp(-0.30/C) \exp(-3850/T)$ , with T in K."""
moisture = np.asarray(moisture, dtype=float)
temperature_k = np.asarray(temperature_k, dtype=float)
if np.any(moisture <= 0.0) or np.any(temperature_k <= 0.0):
    raise ValueError("Positive moisture and absolute temperature are required.")
return 4.2e-4 * np.exp(-0.30 / moisture) * np.exp(-3850.0 / temperature_k)

def make_material_grid(nominal_initial_step_cm: float) -> MaterialGrid:
    step_m = nominal_initial_step_cm / 100.0
    intervals = int(round(INITIAL_RADIUS_M / step_m))
    if not math.isclose(intervals * step_m, INITIAL_RADIUS_M, abs_tol=1.0e-12):
        raise ValueError("The nominal initial radial step must divide 2 cm exactly.")
    logical_faces = np.linspace(0.0, 1.0, intervals + 1)
    faces_xi = 1.0 - (1.0 - logical_faces) ** 2
    return MaterialGrid(
        centers_xi=0.5 * (faces_xi[:-1] + faces_xi[1:]),
        west_faces_xi=faces_xi[:-1],
        east_faces_xi=faces_xi[1:],
    )

def physical_grid(material_grid: MaterialGrid, radius_m: float) -> q2.Grid:
    west = radius_m * material_grid.west_faces_xi
    east = radius_m * material_grid.east_faces_xi
    centers = radius_m * material_grid.centers_xi
    volumes = math.pi * (east**2 - west**2)
    return q2.Grid(centers, west, east, volumes)

def build_dynamic_diffusion_system(
    grid: q2.Grid,
    radius_m: float,
    field_previous: np.ndarray,
    diffusion_coefficient: np.ndarray,
    storage_coefficient: np.ndarray,
    boundary_transfer_coefficient: float,
    boundary_value: float,
    time_step_s: float,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, float]:
    """Backward-Euler FVM system on the current shrunk physical geometry."""
    conductances = q2.internal_conductances(grid, diffusion_coefficient)
    storage = storage_coefficient * grid.volumes_m3_m / time_step_s
    lower = -conductances.copy()
    upper = -conductances.copy()
    diagonal = storage.copy()
    diagonal[:-1] += conductances

```

```

diagonal[1:] += conductances
rhs = storage * field_previous

surface_area_per_length = 2.0 * math.pi * radius_m
boundary_conductance = surface_area_per_length / (
    (radius_m - grid.radii_m[-1]) / diffusion_coefficient[-1]
    + 1.0 / boundary_transfer_coefficient
)
diagonal[-1] += boundary_conductance
rhs[-1] += boundary_conductance * boundary_value
return lower, diagonal, upper, rhs, boundary_conductance

def reconstruct_center(grid: q2.Grid, values: np.ndarray) -> float:
    """Second-order reconstruction using even symmetry at r=0."""
    r0_squared = grid.radii_m[0] ** 2
    r1_squared = grid.radii_m[1] ** 2
    return float(
        (values[0] * r1_squared - values[1] * r0_squared)
        / (r1_squared - r0_squared)
    )

def advance_coupled_step(
    material_grid: MaterialGrid,
    radius_m: float,
    temperature_previous_k: np.ndarray,
    moisture_previous: np.ndarray,
    oven_temperature_k: float,
    oven_moisture: float,
    time_step_s: float,
    temperature_tolerance_k: float = 1.0e-8,
    moisture_tolerance: float = 1.0e-10,
    maximum_iterations: int = 30,
) -> StepResult:
    """Advance one material-coordinate time step by Picard and Thomas solves."""
    grid = physical_grid(material_grid, radius_m)
    temperature_iterate = temperature_previous_k.copy()
    moisture_iterate = moisture_previous.copy()
    for iteration in range(1, maximum_iterations + 1):
        conductivity = conductivity_w_m_k(moisture_iterate)
        heat_storage = density_kg_m3(moisture_iterate) * heat_capacity_j_kg_k(
            moisture_iterate
        )
    heat_system = build_dynamic_diffusion_system(
        grid,
        radius_m,

```

```

        temperature_previous_k,
        conductivity,
        heat_storage,
        HEAT_TRANSFER_COEFFICIENT_W_M2_K,
        oven_temperature_k,
        time_step_s,
    )
    temperature_updated = q2.solve_tridiagonal(*heat_system[:4])

    diffusivity = moisture_diffusivity_m2_s(
        moisture_iterate, temperature_updated
    )
    moisture_system = build_dynamic_diffusion_system(
        grid,
        radius_m,
        moisture_previous,
        diffusivity,
        np.ones_like(moisture_iterate),
        MASS_TRANSFER_COEFFICIENT_M_S,
        oven_moisture,
        time_step_s,
    )
    moisture_updated = q2.solve_tridiagonal(*moisture_system[:4])

    temperature_error = float(
        np.max(np.abs(temperature_updated - temperature_iterate))
    )
    moisture_error = float(np.max(np.abs(moisture_updated - moisture_iterate)))
    temperature_iterate = temperature_updated
    moisture_iterate = moisture_updated
    if (
        temperature_error < temperature_tolerance_k
        and moisture_error < moisture_tolerance
    ):
        break
    else:
        raise RuntimeError("Coupled Picard iteration did not converge.")

    final_conductivity = conductivity_w_m_k(moisture_iterate)
    final_diffusivity = moisture_diffusivity_m2_s(
        moisture_iterate, temperature_iterate
    )
    surface_distance = radius_m - grid.radii_m[-1]
    surface_temperature = q2.reconstruct_surface_value(
        temperature_iterate[-1],
        oven_temperature_k,
        final_conductivity[-1],

```

```

        HEAT_TRANSFER_COEFFICIENT_W_M2_K,
        surface_distance,
    )
    surface_moisture = q2.reconstruct_surface_value(
        moisture_iterate[-1],
        oven_moisture,
        final_diffusivity[-1],
        MASS_TRANSFER_COEFFICIENT_M_S,
        surface_distance,
    )
    final_moisture_system = build_dynamic_diffusion_system(
        grid,
        radius_m,
        moisture_previous,
        final_diffusivity,
        np.ones_like(moisture_iterate),
        MASS_TRANSFER_COEFFICIENT_M_S,
        oven_moisture,
        time_step_s,
    )
    boundary_conductance = final_moisture_system[-1]
    storage_change = float(
        np.sum(grid.volumes_m3_m * (moisture_iterate - moisture_previous))
    )
    return StepResult(
        temperature_k=temperature_iterate,
        moisture_kg_kg=moisture_iterate,
        surface_temperature_k=float(surface_temperature),
        surface_moisture_kg_kg=float(surface_moisture),
        moisture_boundary_conductance=float(boundary_conductance),
        moisture_storage_change=storage_change,
        minimum_diffusivity_m2_s=float(np.min(final_diffusivity)),
        maximum_diffusivity_m2_s=float(np.max(final_diffusivity)),
        iterations=iteration,
    )

def sample_physical_field(
    material_grid: MaterialGrid,
    radius_m: float,
    values: np.ndarray,
    surface_value: float,
    query_radii_cm: np.ndarray,
) -> np.ndarray:
    """Sample current physical radii; return NaN where a point is outside."""
    grid = physical_grid(material_grid, radius_m)
    extended_radii = np.concatenate(([0.0], grid.radii_m, [radius_m]))

```

```

extended_values = np.concatenate(
    ([reconstruct_center(grid, values)], values, [surface_value])
)
query_m = np.asarray(query_radii_cm, dtype=float) / 100.0
sampled = np.full(query_m.shape, np.nan, dtype=float)
valid = query_m <= radius_m + 1.0e-12
sampled[valid] = np.interp(
    np.minimum(query_m[valid], radius_m), extended_radii, extended_values
)
return sampled

def maximum_with_location(
    material_grid: MaterialGrid,
    radius_m: float,
    moisture: np.ndarray,
    surface_moisture: float,
) -> tuple[float, float]:
    grid = physical_grid(material_grid, radius_m)
    values = np.concatenate(
        ([reconstruct_center(grid, moisture)], moisture, [surface_moisture])
    )
    radii_cm = np.concatenate(
        ([0.0], grid.radii_m * 100.0, [radius_m * 100.0])
    )
    index = int(np.argmax(values))
    return float(values[index]), float(radii_cm[index])

def choose_time_step(
    time_s: float,
    maximum_moisture: float,
    early_step_s: float,
    late_step_s: float,
    final_step_s: float,
) -> tuple[float, str]:
    if time_s < MEASURED_ENVIRONMENT_END_S:
        return early_step_s, "measured_environment"
    if maximum_moisture < DRYING_THRESHOLD_KG_KG + 0.002:
        return final_step_s, "threshold_refinement"
    return late_step_s, "constant_environment"

def simulate_until_dry(
    boundary: BoundaryProgram,
    radius_program: RadiusProgram,
    nominal_initial_step_cm: float,

```

```

    early_step_s: float,
    late_step_s: float,
    final_step_s: float,
) -> ShrinkageResult:
    material_grid = make_material_grid(nominal_initial_step_cm)
    n = material_grid.centers_xi.size
    temperature = np.full(n, INITIAL_TEMPERATURE_K, dtype=float)
    moisture = np.full(n, INITIAL_MOISTURE_KG_KG, dtype=float)
    surface_temperature = INITIAL_TEMPERATURE_K
    surface_moisture = INITIAL_MOISTURE_KG_KG
    time_s = 0.0
    next_output_s = OUTPUT_INTERVAL_S

    record_times: list[float] = []
    record_radii: list[float] = []
    temperature_records: list[np.ndarray] = []
    moisture_records: list[np.ndarray] = []
    surface_temperature_records: list[float] = []
    surface_moisture_records: list[float] = []

    maximum_moisture = INITIAL_MOISTURE_KG_KG
    controlling_radius_cm = 0.0
    previous_time_s = 0.0
    previous_maximum = maximum_moisture
    maximum_picard_iterations = 0
    time_step_counts = {
        "measured_environment": 0,
        "constant_environment": 0,
        "threshold_refinement": 0,
        "crossing_recomputed": 0,
    }
    discrete_storage_change = 0.0
    integrated_outflow = 0.0
    minimum_diffusivity = math.inf
    maximum_diffusivity = 0.0
    forced_final_resolution = False

    while time_s < MAXIMUM_SIMULATION_TIME_S:
        if forced_final_resolution:
            proposed_step = final_step_s
            step_label = "threshold_refinement"
        else:
            proposed_step, step_label = choose_time_step(
                time_s,
                maximum_moisture,
                early_step_s,
                late_step_s,

```

```

        final_step_s,
    )
    time_step_s = proposed_step
    if (
        time_s < MEASURED_ENVIRONMENT_END_S
        < time_s + time_step_s
    ):
        time_step_s = MEASURED_ENVIRONMENT_END_S - time_s
    if time_s < next_output_s < time_s + time_step_s:
        time_step_s = next_output_s - time_s
    if time_s + time_step_s > MAXIMUM_SIMULATION_TIME_S:
        time_step_s = MAXIMUM_SIMULATION_TIME_S - time_s
    if time_step_s <= 0.0:
        raise RuntimeError("Non-positive adaptive time step encountered.")

    new_time_s = time_s + time_step_s
    new_radius_m = radius_program.radius_m(new_time_s)
    oven_temperature_k, oven_moisture = boundary.values(new_time_s)
    step = advance_coupled_step(
        material_grid,
        new_radius_m,
        temperature,
        moisture,
        oven_temperature_k,
        oven_moisture,
        time_step_s,
    )
    new_maximum, new_controlling_radius = maximum_with_location(
        material_grid,
        new_radius_m,
        step.moisture_kg_kg,
        step.surface_moisture_kg_kg,
    )
    if new_maximum < DRYING_THRESHOLD_KG_KG and time_step_s > final_step_s:
        forced_final_resolution = True
        time_step_counts["crossing_recomputed"] += 1
        continue

    discrete_storage_change += step.moisture_storage_change
    integrated_outflow += (
        time_step_s
        * step.moisture_boundary_conductance
        * (step.moisture_kg_kg[-1] - oven_moisture)
    )
    temperature = step.temperature_k
    moisture = step.moisture_kg_kg
    surface_temperature = step.surface_temperature_k

```

```

surface_moisture = step.surface_moisture_kg_kg
time_s = new_time_s
maximum_moisture = new_maximum
controlling_radius_cm = new_controlling_radius
maximum_picard_iterations = max(
    maximum_picard_iterations, step.iterations
)
minimum_diffusivity = min(minimum_diffusivity, step.minimum_diffusivity_m2_s)
maximum_diffusivity = max(maximum_diffusivity, step.maximum_diffusivity_m2_s)
time_step_counts[step_label] += 1

is_output_time = math.isclose(time_s, next_output_s, abs_tol=1.0e-8)
if is_output_time:
    current_radius_cm = new_radius_m * 100.0
    record_times.append(time_s)
    record_radii.append(current_radius_cm)
    temperature_records.append(
        sample_physical_field(
            material_grid,
            new_radius_m,
            temperature,
            surface_temperature,
            OUTPUT_RADII_CM,
        )
    )
    moisture_records.append(
        sample_physical_field(
            material_grid,
            new_radius_m,
            moisture,
            surface_moisture,
            OUTPUT_RADII_CM,
        )
    )
    surface_temperature_records.append(surface_temperature)
    surface_moisture_records.append(surface_moisture)
    next_output_s += OUTPUT_INTERVAL_S

if maximum_moisture < DRYING_THRESHOLD_KG_KG:
    if not is_output_time:
        current_radius_cm = new_radius_m * 100.0
        record_times.append(time_s)
        record_radii.append(current_radius_cm)
        temperature_records.append(
            sample_physical_field(
                material_grid,
                new_radius_m,

```

```

        temperature,
        surface_temperature,
        OUTPUT_RADII_CM,
    )
)
moisture_records.append(
    sample_physical_field(
        material_grid,
        new_radius_m,
        moisture,
        surface_moisture,
        OUTPUT_RADII_CM,
    )
)
surface_temperature_records.append(surface_temperature)
surface_moisture_records.append(surface_moisture)
break

previous_time_s = time_s
previous_maximum = maximum_moisture
else:
    raise RuntimeError("Maximum simulation time reached before drying.")

return ShrinkageResult(
    times_s=np.asarray(record_times, dtype=float),
    radii_cm=np.asarray(record_radii, dtype=float),
    temperature_k=np.vstack(temperature_records),
    moisture_kg_kg=np.vstack(moisture_records),
    surface_temperature_k=np.asarray(surface_temperature_records, dtype=float),
    surface_moisture_kg_kg=np.asarray(surface_moisture_records, dtype=float),
    drying_time_s=time_s,
    previous_time_s=previous_time_s,
    previous_maximum_moisture=previous_maximum,
    final_maximum_moisture=maximum_moisture,
    controlling_radius_cm=controlling_radius_cm,
    final_internal_temperature_k=temperature,
    final_internal_moisture_kg_kg=moisture,
    maximum_picard_iterations=maximum_picard_iterations,
    time_step_counts=time_step_counts,
    discrete_moisture_storage_change=discrete_storage_change,
    integrated_moisture_outflow=integrated_outflow,
    minimum_diffusivity_m2_s=minimum_diffusivity,
    maximum_diffusivity_m2_s=maximum_diffusivity,
)

def record_indices_at_times(

```

```

        result: ShrinkageResult, query_times_s: np.ndarray
    ) -> np.ndarray:
        lookup = {round(float(time), 8): index for index, time in enumerate(result.times_s)}
        indices: list[int] = []
        for time in query_times_s:
            key = round(float(time), 8)
            if key not in lookup:
                raise ValueError(f"Requested time {time:g} s was not recorded.")
            indices.append(lookup[key])
        return np.asarray(indices, dtype=int)

def build_table_times(drying_time_s: float) -> np.ndarray:
    regular = np.arange(
        TABLE_INTERVAL_S,
        drying_time_s - 1.0e-9,
        TABLE_INTERVAL_S,
        dtype=float,
    )
    if regular.size and math.isclose(regular[-1], drying_time_s, abs_tol=1.0e-8):
        return regular
    return np.append(regular, drying_time_s)

def write_full_csv(path: Path, result: ShrinkageResult) -> None:
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(
            [
                "time_s",
                "radius_cm",
                *[f"r_{radius:.1f}_cm" for radius in OUTPUT_RADII_CM],
                "surface",
            ]
        )
        for time_s, radius_cm, values, surface in zip(
            result.times_s,
            result.radii_cm,
            result.moisture_kg_kg,
            result.surface_moisture_kg_kg,
        ):
            writer.writerow(
                [
                    f"{time_s:.0f}",
                    f"{radius_cm:.6f}",
                    *[" " if np.isnan(value) else f"{value:.4f}" for value in values],
                    f"{surface:.4f}",
                ]
            )

```

```

        ]
    )

def write_temperature_csv(path: Path, result: ShrinkageResult) -> None:
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(
            [
                "time_s",
                "radius_cm",
                *[f"r_{radius:.1f}_cm" for radius in OUTPUT_RADII_CM],
                "surface_temperature_K",
            ]
        )
        for time_s, radius_cm, values, surface in zip(
            result.times_s,
            result.radii_cm,
            result.temperature_k,
            result.surface_temperature_k,
        ):
            writer.writerow(
                [
                    f"{time_s:.0f}",
                    f"{radius_cm:.6f}",
                    *[" " if np.isnan(value) else f"{value:.4f}" for value in values],
                    f"{surface:.4f}",
                ]
            )

def write_table_csv(
    path: Path,
    result: ShrinkageResult,
    table_times_s: np.ndarray,
) -> None:
    indices = record_indices_at_times(result, table_times_s)
    output_columns = [
        int(np.where(np.isclose(OUTPUT_RADII_CM, radius))[0][0])
        for radius in TABLE_RADII_CM
    ]
    with path.open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(
            ["时间/h", *[f"{radius:g} cm" for radius in TABLE_RADII_CM], "药材表面"]
        )
        for time_s, index in zip(table_times_s, indices):

```

```

        label = (
            "烘干结束时间"
            if math.isclose(time_s, result.drying_time_s, abs_tol=1.0e-8)
            else f"{time_s / 3600.0:.0f}"
        )
        values = result.moisture_kg_kg[index, output_columns]
        writer.writerow(
            [
                label,
                *[" if np.isnan(value) else f'{value:.4f}' for value in values],
                f"{result.surface_moisture_kg_kg[index]:.4f}",
            ]
        )

def write_workbook_payload(path: Path, result: ShrinkageResult) -> None:
    rows: list[list[float | None]] = []
    for time_s, values, surface in zip(
        result.times_s, result.moisture_kg_kg, result.surface_moisture_kg_kg
    ):
        rows.append(
            [
                int(round(float(time_s))),
                *[" if np.isnan(value) else round(float(value), 4) for value in values],
                round(float(surface), 4),
            ]
        )
    payload = {
        "headers": [
            "时间\\到药材中心的距离",
            *[round(float(radius), 1) for radius in OUTPUT_RADII_CM],
            "药材表面",
        ],
        "rows": rows,
        "drying_time_s": result.drying_time_s,
    }
    with path.open("w", encoding="utf-8") as stream:
        json.dump(payload, stream, ensure_ascii=False, separators=(",", ":"))

def common_refinement_difference(
    production: ShrinkageResult, coarse: ShrinkageResult
) -> tuple[int, float, float]:
    coarse_lookup = {
        round(float(time), 8): index for index, time in enumerate(coarse.times_s)
    }
    differences: list[np.ndarray] = []

```

```

common_rows = 0
for index, time in enumerate(production.times_s):
    key = round(float(time), 8)
    if key not in coarse_lookup:
        continue
    coarse_index = coarse_lookup[key]
    a = production.moisture_kg_kg[index]
    b = coarse.moisture_kg_kg[coarse_index]
    mask = np.isfinite(a) & np.isfinite(b)
    if np.any(mask):
        differences.append(np.abs(a[mask] - b[mask]))
        differences.append(
            np.asarray(
                [
                    abs(
                        production.surface_moisture_kg_kg[index]
                        - coarse.surface_moisture_kg_kg[coarse_index]
                    )
                ]
            )
        )
    common_rows += 1
combined = np.concatenate(differences)
return common_rows, float(np.max(combined)), float(np.mean(combined))

def build_validation(
    boundary: BoundaryProgram,
    radius_program: RadiusProgram,
    production: ShrinkageResult,
    coarse: ShrinkageResult,
) -> dict:
    common_rows, refinement_maximum, refinement_mean = common_refinement_difference(
        production, coarse
    )
    residual = (
        production.discrete_moisture_storage_change
        + production.integrated_moisture_outflow
    )
    balance_scale = max(
        abs(production.discrete_moisture_storage_change),
        abs(production.integrated_moisture_outflow),
        1.0e-30,
    )

    invalid_distance_values = 0
    missing_inside_values = 0

```

```

radial_order_violations = 0
time_order_violations = 0
for row_index, radius_cm in enumerate(production.radii_cm):
    inside = OUTPUT_RADII_CM <= radius_cm + 1.0e-10
    invalid_distance_values += int(
        np.count_nonzero(np.isfinite(production.moisture_kg_kg[row_index, ~inside]))
    )
    missing_inside_values += int(
        np.count_nonzero(~np.isfinite(production.moisture_kg_kg[row_index, inside]))
    )
    radial_values = np.concatenate(
        (
            production.moisture_kg_kg[row_index, inside],
            [production.surface_moisture_kg_kg[row_index]],
        )
    )
    radial_order_violations += int(np.count_nonzero(np.diff(radial_values) > 1.0e-8))
for column in range(OUTPUT_RADII_CM.size):
    series = production.moisture_kg_kg[:, column]
    valid = np.isfinite(series)
    if np.count_nonzero(valid) > 1:
        time_order_violations += int(
            np.count_nonzero(np.diff(series[valid]) > 1.0e-8)
        )
time_order_violations += int(
    np.count_nonzero(np.diff(production.surface_moisture_kg_kg) > 1.0e-8)
)

source_reconstruction_error = float(
    np.max(
        np.abs(
            np.asarray(
                [radius_program.radius_cm(t) for t in radius_program.source_times_s]
            )
            - radius_program.source_radii_cm
        )
    )
)

finite_temperature = production.temperature_k[np.isfinite(production.temperature_k)]
finite_moisture = production.moisture_kg_kg[np.isfinite(production.moisture_kg_kg)]
return {
    "model_scope": {
        "geometry": "one-dimensional axisymmetric cylinder with moving radius",
        "coordinate": "uniform material coordinate xi=r/R(t)",
        "radius_interpolation": "piecewise linear Attachment 2 data; last value held only if needed",
        "constitutive_laws": "Appendix 4 used over the full process",
    }
}

```

```

"temperature_state_unit": "K",
"latent_heat": "excluded consistently with the simpler questions 1-3 model",
"drying_threshold_kg_kg": DRYING_THRESHOLD_KG_KG,
"environment_after_4h": "time-weighted mean over Attachment 1 final hour",
"result4_distance_rule": (
    "fixed physical radii 0.0-2.0 cm every 0.1 cm plus the moving "
    "surface; a fixed-radius cell is blank when its radius exceeds R(t)"
),
"production_initial_nominal_step_cm": 0.00625,
"production_time_steps_s": [1.0, 30.0, 1.0],
"coarse_initial_nominal_step_cm": 0.0125,
"coarse_time_steps_s": [1.0, 30.0, 1.0],
"saved_interval_s": OUTPUT_INTERVAL_S,
},
"constant_boundary": {
    "temperature_K": boundary.plateau_temperature_k,
    "temperature_C_for_reference": boundary.plateau_temperature_k - 273.15,
    "moisture_kg_kg": boundary.plateau_moisture_kg_kg,
},
"radius": {
    "attachment_interval_s": [
        float(radius_program.source_times_s[0]),
        float(radius_program.source_times_s[-1]),
    ],
    "initial_cm": float(radius_program.source_radii_cm[0]),
    "drying_time_cm": float(production.radii_cm[-1]),
    "source_node_reconstruction_max_error_cm": source_reconstruction_error,
    "recorded_nonincreasing": bool(np.all(np.diff(production.radii_cm) <= 1.0e-12)),
},
"drying_time": {
    "production_s": production.drying_time_s,
    "production_h": production.drying_time_s / 3600.0,
    "coarse_s": coarse.drying_time_s,
    "coarse_h": coarse.drying_time_s / 3600.0,
    "coarse_production_difference_s": abs(
        production.drying_time_s - coarse.drying_time_s
    ),
    "previous_time_s": production.previous_time_s,
    "previous_maximum_moisture": production.previous_maximum_moisture,
    "final_maximum_moisture": production.final_maximum_moisture,
    "controlling_radius_cm": production.controlling_radius_cm,
},
"grid_refinement": {
    "common_record_count": common_rows,
    "maximum_moisture_change_kg_kg": refinement_maximum,
    "mean_moisture_change_kg_kg": refinement_mean,
},

```

```

"discrete_moisture_balance": {
    "summed_storage_change": production.discrete_moisture_storage_change,
    "integrated_boundary_outflow": production.integrated_moisture_outflow,
    "residual": residual,
    "relative_residual": abs(residual) / balance_scale,
},
"physical_ranges": {
    "temperature_min_K": float(np.min(finite_temperature)),
    "temperature_max_K": float(np.max(finite_temperature)),
    "moisture_min_kg_kg": float(np.min(finite_moisture)),
    "moisture_max_kg_kg": float(np.max(finite_moisture)),
    "surface_moisture_min_kg_kg": float(np.min(production.surface_moisture_kg_kg)),
    "diffusivity_min_m2_s": production.minimum_diffusivity_m2_s,
    "diffusivity_max_m2_s": production.maximum_diffusivity_m2_s,
},
"checks": {
    "previous_state_not_dry": bool(
        production.previous_maximum_moisture >= DRYING_THRESHOLD_KG_KG
    ),
    "final_state_all_dry": bool(
        production.final_maximum_moisture < DRYING_THRESHOLD_KG_KG
    ),
    "invalid_values_outside_current_radius": invalid_distance_values,
    "missing_values_inside_current_radius": missing_inside_values,
    "radial_order_violation_count": radial_order_violations,
    "time_monotonicity_violation_count": time_order_violations,
    "maximum_picard_iterations": production.maximum_picard_iterations,
    "time_step_counts": production.time_step_counts,
},
}

```

```
def main() -> None:
```

```

    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--attachment1", type=Path, help="Path to Attachment 1")
    parser.add_argument("--attachment2", type=Path, help="Path to Attachment 2")
    parser.add_argument("--repo-root", type=Path, help="Repository root")
    parser.add_argument(
        "--result-dir",
        type=Path,
        help="Output directory (default: result/A_q4_shrinkage)",
    )
    args = parser.parse_args()

    script_path = Path(__file__).resolve()
    repo_root = args.repo_root.resolve() if args.repo_root else script_path.parents[1]
    attachment1 = (

```

```

    args.attachment1.resolve()
    if args.attachment1
    else find_attachment(repo_root, "附件1.xlsx")
)
attachment2 = (
    args.attachment2.resolve()
    if args.attachment2
    else find_attachment(repo_root, "附件2.xlsx")
)
result_dir = (
    args.result_dir.resolve()
    if args.result_dir
    else repo_root / "result" / "A_q4_shrinkage"
)
result_dir.mkdir(parents=True, exist_ok=True)

environment = q2.load_and_preprocess_environment(attachment1)
boundary = build_boundary_program(environment)
radius_program = load_radius_program(attachment2)

coarse = simulate_until_dry(
    boundary,
    radius_program,
    nominal_initial_step_cm=0.0125,
    early_step_s=1.0,
    late_step_s=30.0,
    final_step_s=1.0,
)
production = simulate_until_dry(
    boundary,
    radius_program,
    nominal_initial_step_cm=0.00625,
    early_step_s=1.0,
    late_step_s=30.0,
    final_step_s=1.0,
)

table_times = build_table_times(production.drying_time_s)
write_full_csv(result_dir / "moisture_full_60s_0p1cm.csv", production)
write_temperature_csv(
    result_dir / "temperature_auxiliary_60s_0p1cm_K.csv", production
)
write_table_csv(result_dir / "table6_moisture.csv", production, table_times)
write_workbook_payload(result_dir / "result4_payload.json", production)
validation = build_validation(boundary, radius_program, production, coarse)
with (result_dir / "validation_summary.json").open("w", encoding="utf-8") as stream:
    json.dump(validation, stream, ensure_ascii=False, indent=2)

```

```

print(
    f"Drying time: {production.drying_time_s:.0f} s "
    f"= {production.drying_time_s / 3600.0:.6f} h"
)
print(
    f"Threshold bracket: {production.previous_time_s:.0f} -> "
    f"{production.drying_time_s:.0f} s; "
    f"{production.previous_maximum_moisture:.10f} -> "
    f"{production.final_maximum_moisture:.10f} kg/kg"
)
print(json.dumps(validation, ensure_ascii=False, indent=2))
print(f"Results: {result_dir}")

if __name__ == "__main__":
    main()

```